

فصل ۸: بن‌بست‌ها (Deadlocks)

اهداف فصل (Objectives)

در این فصل، با مفاهیم زیر آشنا می‌شویم:

- نحوه‌ی رخ دادن بن‌بست در صورت استفاده از قفل‌های متقابل (mutex locks)
- شرایط لازم برای وقوع بن‌بست
- تشخیص وضعیت بن‌بست با استفاده از نمودار تخصیص منابع
- بررسی روش‌های مختلف برای مدیریت بن‌بست شامل:
 - جلوگیری از بن‌بست (Prevention)
 - اجتناب از بن‌بست (Avoidance)
 - تشخیص بن‌بست (Detection)
 - بازیابی از بن‌بست (Recovery)
- کاربرد الگوریتم بانکدار (Banker's Algorithm) برای اجتناب از بن‌بست

مدل سیستم (System Model)

برای درک بن‌بست‌ها، ابتدا باید مدل ساده‌شده‌ای از سیستم را بشناسیم که شامل موارد زیر است:

- فرآیندها (Processes):** برنامه‌های در حال اجرا که به منابع نیاز دارند.
- منابع (Resources):** مانند چاپگر، فایل، قفل، CPU و ... که فرآیندها به‌طور انحصاری آن‌ها را می‌گیرند و آزاد می‌کنند.
- درخواست و تخصیص منابع:** فرآیندها منابع را درخواست می‌کنند و در صورت موجود بودن، سیستم عامل آن را تخصیص می‌دهد.

تعریف بن‌بست (Deadlock)

بن‌بست زمانی رخ می‌دهد که **گروهی از فرآیندها منتظر منابعی هستند که در اختیار یکدیگر هستند** و هیچ‌کدام نمی‌توانند ادامه دهند. یعنی همه منتظرند، ولی هیچ‌کس نمی‌تواند پیش برود.

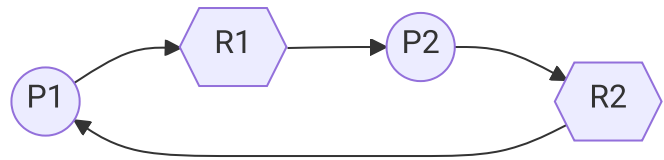
شرایط لازم برای وقوع بن‌بست

چهار شرط زیر باید همزمان برقرار باشند تا بن‌بست رخ دهد:

- تخصیص انحصاری (Mutual Exclusion):** منبع فقط به یک فرآیند داده می‌شود.
- نگهداری و انتظار (Hold and Wait):** فرآیند یک منبع را نگه داشته و منتظر منابع دیگر است.
- عدم پیش‌گیری (No Preemption):** منابع را نمی‌توان از فرآیند گرفت، مگر اینکه خودش آزاد کند.
- انتظار چرخشی (Circular Wait):** زنجیره‌ای از فرآیندها وجود دارد که هرکدام منتظر منبعی هستند که در اختیار دیگری است.

نمودار تخصیص منابع (Resource Allocation Graph)

می‌توان بن‌بست را با استفاده از نمودار زیر مدل کرد:



در این نمودار:

- دایره‌ها فرآیندها هستند (P1, P2).
- لوزی‌ها منابع هستند (R1, R2).
- فلش از فرآیند به منبع: درخواست منبع.
- فلش از منبع به فرآیند: منبع در اختیار آن فرآیند است.

در این مثال، یک چرخه بین فرآیندها و منابع وجود دارد → احتمال بن‌بست.

مثال ساده آموزشی 🎓

فرض کنید:

- فرآیند A یک چاپگر گرفته و منتظر دسترسی به فایل است.
- فرآیند B فایل را گرفته و منتظر چاپگر است.

هر دو منتظر همدیگرند و هیچ‌کدام آزاد نمی‌کنند → **بن‌بست**.

روش‌های مدیریت بن‌بست‌ها 🛠️

در اسلایدها به چهار راهکار اشاره شده:

1. **پیشگیری (Prevention)**: حذف یکی از شرایط لازم.
2. **اجتناب (Avoidance)**: استفاده از الگوریتم‌هایی مثل بانکدار برای بررسی حالت امن.
3. **تشخیص (Detection)**: بررسی وجود چرخه و اعلام بن‌بست.
4. **بازیابی (Recovery)**: آزادسازی منابع یا پایان دادن به فرآیندها برای خارج شدن از بن‌بست.

نکات کلیدی ✅

- بن‌بست زمانی رخ می‌دهد که چند فرآیند منابع را به‌صورت متقابل نگه داشته و منتظر منابع دیگر باشند.
- چهار شرط برای وقوع بن‌بست باید **همزمان برقرار باشند**.
- نمودار تخصیص منابع، ابزاری برای تحلیل وقوع یا احتمال وقوع بن‌بست است.
- چهار رویکرد برای مقابله با بن‌بست‌ها وجود دارد که در ادامه فصل بررسی می‌شوند.

مدل سیستم (System Model)

توضیح کامل و قابل فهم

در مدل سیستم برای بررسی بن‌بست، فرض می‌شود که:

- سیستم شامل مجموعه‌ای از منابع است.
 - این منابع می‌توانند از انواع مختلف باشند:
- مانند **چرخه‌های پردازنده (CPU Cycles)**، **فضای حافظه (Memory Space)**، و **دستگاه‌های ورودی/خروجی (I/O Devices)**.

تعریف نوع منبع (Resource Type)

هر نوع منبع (مثلاً چاپگر، دیسک، پردازنده) با نماد R_i نشان داده می‌شود و می‌تواند چندین نمونه (Instance) داشته باشد. مثلاً:

- R_1 = چاپگر (2 عدد)
- R_2 = اسکنر (1 عدد)

نحوه استفاده فرایندها از منابع

هر فرایند برای استفاده از منابع از سه مرحله زیر عبور می‌کند:

1. درخواست (Request): فرایند از سیستم عامل می‌خواهد که منبع خاصی را در اختیارش بگذارد.
2. استفاده (Use): فرایند از منبع برای انجام وظیفه‌اش استفاده می‌کند.
3. آزادسازی (Release): پس از پایان کار، منبع را آزاد می‌کند تا دیگر فرایندها بتوانند از آن استفاده کنند.

بن‌بست با استفاده از Semaphore (Deadlock with Semaphores)

در این مثال، دو Semaphore داریم:

- مقدار اولیه: S_1 1
- مقدار اولیه: S_2 1

و دو نخ (Thread):

- که به ترتیب زیر عمل می‌کند T_1 :

```
wait(S1);  
wait(S2);
```

- که به صورت زیر عمل می‌کند T_2 :

```
wait(S2);  
wait(S1);
```

توضیح:

- اگر T_1 اول S_1 را بگیرد و منتظر S_2 بماند
- و همزمان T_2 اول S_2 را بگیرد و منتظر S_1 بماند
- => هر دو در وضعیت انتظار یکدیگر گیر می‌کنند و بن‌بست (Deadlock) اتفاق می‌افتد.

مثال ساده آموزشی

فرض کنید:

- سمان S_1 یک کلید برای قفل اتاق A است.
- سمان S_2 کلید قفل اتاق B است.
- شخص ۱ (T_1) ابتدا کلید اتاق A را می‌گیرد و منتظر کلید B می‌ماند.
- شخص ۲ (T_2) ابتدا کلید B را می‌گیرد و منتظر کلید A می‌ماند.

هیچ کدام کلید را پس نمی دهند و هیچ کدام نمی توانند وارد هر دو اتاق شوند $\Rightarrow$ بن بست.

نکات کلیدی

- منابع سیستم می توانند چند نوع و چند نمونه داشته باشند.
- فرایندها منابع را طبق توالی درخواست $\rightarrow$ استفاده $\rightarrow$ آزادسازی مدیریت می کنند.
- اگر منابع به درستی مدیریت نشوند، امکان ایجاد بن بست وجود دارد.
- مثال Semaphores نشان می دهد که حتی در سیستم های هماهنگ شده (مثل قفل گذاری با Semaphore) نیز امکان بن بست وجود دارد.
- ترتیب نامناسب درخواست منابع توسط نخ ها/فرایندها عامل مهمی در بروز بن بست است.

مدل سیستم (System Model)

توضیح کامل مدل سیستم

در این بخش، مدل سیستم که در بررسی بن بست ها استفاده می شود معرفی می شود:

- سیستم شامل انواع منابع (Resource Types) مختلف است که معمولاً با نمادهای $R_1, R_2, \dots, R_m$ نشان داده می شوند.
- منابع می توانند مواردی مانند **چرخه های CPU**، **فضای حافظه**، **دستگاه های ورودی/خروجی** باشند.
- هر نوع منبع R_i دارای W_i نمونه (Instance) است، یعنی چند نمونه از آن منبع در سیستم وجود دارد.
- هر فرایند (Process) برای استفاده از منابع، یک چرخه سه مرحله ای را طی می کند:
 - درخواست (Request)**: فرایند درخواست یک نمونه از منبع را می دهد.
 - استفاده (Use)**: پس از تخصیص، فرایند از منبع استفاده می کند.
 - آزادسازی (Release)**: بعد از اتمام کار، فرایند منبع را آزاد می کند تا سایر فرایندها بتوانند از آن استفاده کنند.

بن بست در استفاده از Semaphore (سِمافور)

سِمافور یک متغیر یا ساختار داده است که برای همزمانی و جلوگیری از شرایط رقابتی (Race Condition) در برنامه های چندنخی (Multithreading) استفاده می شود.

مثال:

- دو Semaphore به نام های S_1 و S_2 داریم که هر کدام مقدار اولیه ۱ دارند (یعنی فقط یک نخ می تواند همزمان آن ها را بگیرد).
- دو نخ (Thread) به نام های T_1 و T_2 داریم که به ترتیب مراحل زیر را اجرا می کنند:

نخ	مراحل اجرا
T_1	$\text{wait}(S_1) \rightarrow \text{wait}(S_2)$
T_2	$\text{wait}(S_2) \rightarrow \text{wait}(S_1)$

و. منتظر ماندن تا آزاد شود Semaphore (Lock) یعنی تلاش برای گرفتن $\text{wait}(S)$.

چطور بن بست رخ می دهد؟

- اگر T_1 ابتدا S_1 را بگیرد، سپس منتظر S_2 باشد،
- و همزمان T_2 ابتدا S_2 را بگیرد و منتظر S_1 باشد،

آنگاه هر دو نخ منتظر منبعی هستند که دیگری گرفته و آزاد نمی کند؛ این **بن بست (Deadlock)** است.

مثال ساده آموزشی

فرض کنید دو دوست دو کتابخانه مختلف دارند و برای انجام پروژه هر کدام باید از هر دو کتابخانه استفاده کنند:

- دوست ۱ ابتدا کتابخانه ۱ را می‌گیرد، اما منتظر کتابخانه ۲ است.
- دوست ۲ ابتدا کتابخانه ۲ را گرفته و منتظر کتابخانه ۱ است.
- هیچ‌کدام حاضر نیستند کتابخانه خود را رها کنند و هر دو منتظر دیگری مانده‌اند، که این وضعیت نمونه‌ای از بن‌بست است.

نکات کلیدی

- سیستم منابع متعددی دارد که هر نوع منبع چند نمونه می‌تواند داشته باشد.
- هر فرایند منابع را به صورت درخواست، استفاده و آزادسازی مدیریت می‌کند.
- ابزاری برای مدیریت دسترسی همزمان به منابع است Semaphore.
- اگر نخ‌ها یا فرایندها به ترتیب متفاوتی برای قفل کردن منابع اقدام کنند، ممکن است بن‌بست رخ دهد.
- بن‌بست در سیمافورها معمولاً به دلیل انتظار متقابل دایره‌ای ایجاد می‌شود.

ویژگی‌های بن‌بست (Deadlock Characterization)

توضیح کامل

بن‌بست زمانی ممکن است در سیستم رخ دهد که **چهار شرط زیر به صورت همزمان برقرار باشند**. اگر حتی یکی از این شرایط نقض شود، بن‌بست رخ نمی‌دهد. این چهار شرط عبارت‌اند از:

1. انحصار متقابل (Mutual Exclusion)

در هر لحظه فقط **یک نخ** می‌تواند از یک منبع استفاده کند. اگر منبع در حال استفاده باشد، دیگران باید منتظر بمانند.

2. نگهداری و انتظار (Hold and Wait)

نخی که یک یا چند منبع را در اختیار دارد، می‌تواند همزمان **درخواست منابع دیگری** نیز بدهد و منتظر آن‌ها بماند.

3. عدم پیش‌دستی (No Preemption)

سیستم نمی‌تواند منابع را به اجبار از نخ بگیرد. فقط نخ دارنده‌ی منبع می‌تواند **داوطلبانه** پس از اتمام کار، منبع را آزاد کند.

4. انتظار چرخشی (Circular Wait)

مجموعه‌ای از نخ‌ها وجود دارند که هر کدام منتظر منبعی هستند که نخ بعدی در چرخه آن را در اختیار دارد. این چرخه از انتظار باعث بن‌بست می‌شود.

نمودار تخصیص منابع (Resource Allocation Graph)

برای بررسی و شناسایی بن‌بست‌ها، از **نمودار تخصیص منابع** استفاده می‌شود.

اجزای اصلی نمودار:

• مجموعه نخ‌ها (Threads):

$$T = \{T_1, T_2, \dots, T_n\}$$

• مجموعه منابع (Resources):

$$R = \{R_1, R_2, \dots, R_m\}$$

• انواع یال‌ها (Edges):

- **یال درخواست (Request Edge):**

از نخ به منبع: $T_i \rightarrow R_j$

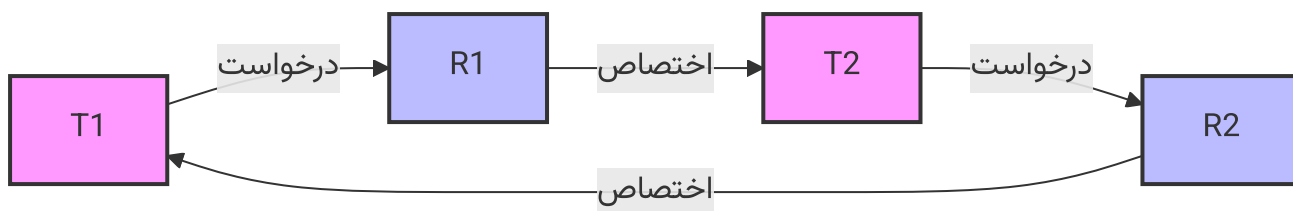
یعنی T_i درخواست R_j را داده است.

- **یال تخصیص (Assignment Edge):**

از منبع به نخ: $R_j \rightarrow T_i$

یعنی منبع R_j به نخ T_i اختصاص یافته است.

نمودار Mermaid از نمونه چرخه انتظار (بن‌بست)



در این مدل:

- اختصاص دارد T2 است که به R1 در حال درخواست T1.
- اختصاص دارد T1 است که به R2 در حال درخواست T2.
- بنابراین یک چرخه‌ی انتظار بین نخ‌ها و منابع ایجاد شده که **نشان‌دهنده بن‌بست است**.

مثال ساده آموزشی

فرض کنید سه دانشجو می‌خواهند پروژه‌ای تیمی انجام دهند و هر کدام برای کار خود به دو ابزار نیاز دارند (مثلاً لپ‌تاپ، میکروسکوپ و پرینتر).

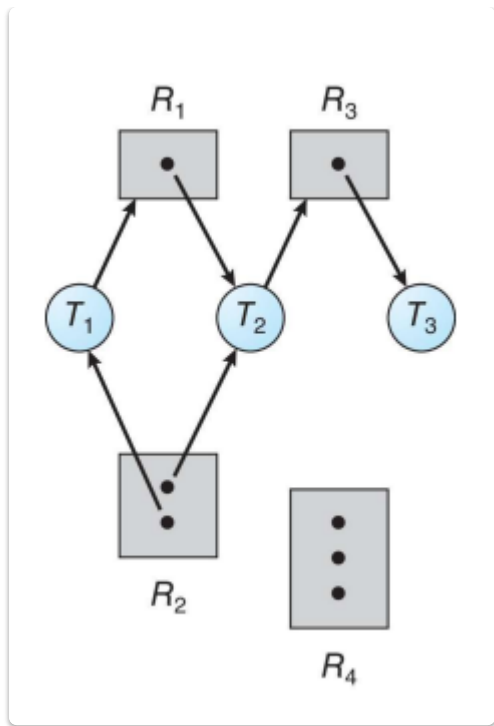
- دانشجو ۱ لپ‌تاپ را دارد و منتظر پرینتر است.
- دانشجو ۲ پرینتر را دارد و منتظر میکروسکوپ است.
- دانشجو ۳ میکروسکوپ را دارد و منتظر لپ‌تاپ است.

در اینجا **شرایط انتظار چرخشی برقرار است** و هیچ‌کدام کار خود را نمی‌توانند ادامه دهند → بن‌بست!

نکات کلیدی

- **بن‌بست زمانی رخ می‌دهد که چهار شرط هم‌زمان برقرار باشند.**
 - انحصار متقابل
 - نگهداری و انتظار
 - عدم پیش‌دستی
 - انتظار چرخشی
- **نمودار تخصیص منابع** ابزار قدرتمندی برای تحلیل و تشخیص بن‌بست است.
- اگر در نمودار تخصیص منابع **چرخه وجود داشته باشد** و هر منبع **فقط یک نمونه** داشته باشد، حتماً بن‌بست رخ داده است.
- اگر حداقل یکی از شرایط چهارگانه نقض شود، **سیستم در برابر بن‌بست ایمن** خواهد بود.

مثال نمودار تخصیص منابع (Resource Allocation Graph Example)



توضیح کامل از تصویر

در این مثال، نمودار تخصیص منابع برای تحلیل وضعیت سیستم و شناسایی احتمال بن‌بست ترسیم شده است. اجزای اصلی شامل نخ‌ها (Threads)، منابع (Resources) و روابط بین آن‌هاست.

اطلاعات اولیه:

- فقط ۱ نمونه R_1 :
- نمونه ۲ R_2 :
- فقط ۱ نمونه R_3 :
- نمونه (در این نمودار استفاده نشده است) ۳ R_4 :

وضعیت نخ‌ها (Threads):

- T_1 :
 - دارای یک نمونه از R_2
 - در حال درخواست R_1
- T_2 :
 - دارای یک نمونه از R_1
 - دارای یک نمونه از R_2
 - در حال درخواست R_3
- T_3 :
 - دارای یک نمونه از R_3

تجزیه نمودار به زبان ساده

در این نمودار:

- $T_1 \rightarrow R_1$: یعنی T_1 منتظر دریافت R_1 است.
- $R_2 \rightarrow T_1$: یعنی T_1 یکی از نمونه‌های R_2 را در اختیار دارد.
- $T_2 \rightarrow R_3$: یعنی T_2 منتظر دریافت R_3 است.
- $R_1 \rightarrow T_2$ و $R_2 \rightarrow T_2$: یعنی T_2 همزمان یکی از نمونه‌های R_1 و یکی از نمونه‌های R_2 را در اختیار دارد.
- $R_3 \rightarrow T_3$: یعنی T_3 تنها استفاده‌کننده از R_3 است.

تحلیل وضعیت بن‌بست

در این وضعیت:

- قرار دارد T_2 است که در اختیار R_1 منتظر T_1 .
 - قرار دارد T_3 است که در اختیار R_3 منتظر T_2 .
 - اما T_3 منتظر هیچ منبعی نیست، پس زنجیره قطع می‌شود و چرخه‌ی کامل انتظار ایجاد نشده.
- نتیجه: در این مثال، بن‌بست وجود ندارد. چون اگرچه T_1 و T_2 در حال انتظار هستند، اما T_3 می‌تواند R_3 را در آینده آزاد کند و سپس منابع به ترتیب آزاد شوند.

مثال ساده آموزشی

فرض کنید سه نفر برای پخت غذا به وسایل زیر نیاز دارند: قابلمه (R_1)، اجاق گاز (R_2)، و چاقو (R_3).

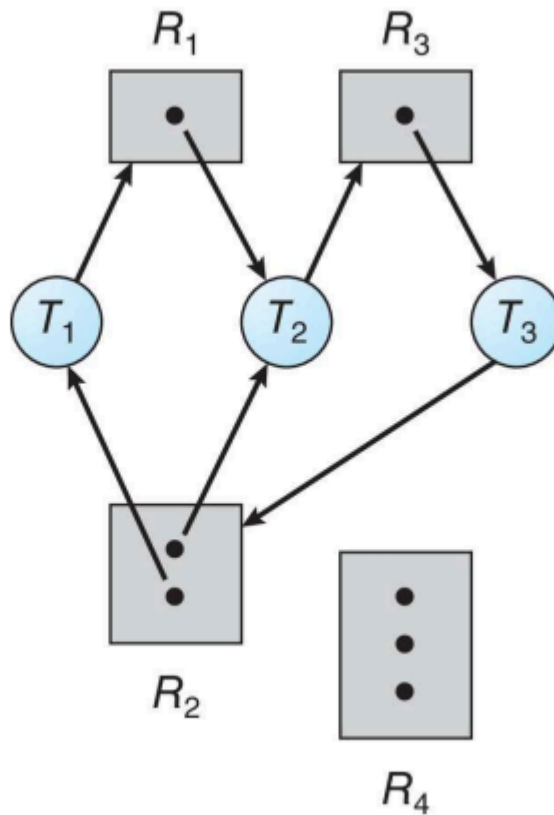
- نفر ۱ (T_1) اجاق را دارد و منتظر قابلمه است.
 - نفر ۲ (T_2) قابلمه و اجاق را دارد و منتظر چاقو است.
 - نفر ۳ (T_3) فقط چاقو را دارد و منتظر هیچ چیز نیست.
- اگر نفر ۳ کارش را تمام کند و چاقو را آزاد کند، نفر ۲ می‌تواند ادامه دهد و سپس نفر ۱.

نکات کلیدی

- نمودار تخصیص منابع نمایشی گرافیکی از رابطه بین نخ‌ها و منابع است.
- یال از نخ به منبع = درخواست
- یال از منبع به نخ = تخصیص
- اگر چرخه‌ی کامل انتظار وجود نداشته باشد، احتمال بن‌بست وجود ندارد.
- تحلیل چنین نموداری کمک می‌کند تا بتوان رفتار سیستم را در شرایط بحرانی بررسی و مدیریت کرد.

نمودار تخصیص منابع همراه با بن‌بست

Resource Allocation Graph with a Deadlock



توضیح کامل از تصویر

این نمودار نمونه‌ای از یک **وضعیت بن‌بست** (Deadlock) را در سیستم نمایش می‌دهد. با دقت به یال‌های درخواست و تخصیص، می‌توان مشاهده کرد که سه نخ به صورت چرخشی در حال انتظار هستند و هیچ‌کدام قادر به ادامه کار نیستند.

اجزای نمودار:

• نخ‌ها (Threads):

- T_1
- T_2
- T_3

• منابع (Resources):

- R_1 (نمونه ۱)
- R_2 (نمونه ۲)
- R_3 (نمونه ۱)
- R_4 (استفاده نشده)

تحلیل دقیق نمودار

تخصیص و درخواست‌ها:

- $T_1 \rightarrow R_1$: T_1 منتظر دریافت R_1
- $R_2 \rightarrow T_1$: T_1 یکی از نمونه‌های R_2 را در اختیار دارد
- $T_2 \rightarrow R_2$: T_2 منتظر دریافت R_2
- $R_1 \rightarrow T_2$: T_2 یکی از نمونه‌های R_1 را در اختیار دارد
- $T_3 \rightarrow R_3$: T_3 منتظر دریافت R_3
- $R_2 \rightarrow T_3$: T_3 یکی از نمونه‌های R_2 را در اختیار دارد
- $R_3 \rightarrow T_2$: T_2 یکی از نمونه‌های R_3 را در اختیار دارد

1. T_2 است $\rightarrow$ در اختیار R_1 منتظر T_1 .
 2. T_3 است $\rightarrow$ در اختیار R_2 منتظر T_2 .
 3. T_2 است $\rightarrow$ در اختیار R_3 منتظر T_3 .
- نتیجه: یک **چرخه‌ی بسته** از انتظار شکل گرفته است. 

✓ در این وضعیت، هیچ نخی نمی‌تواند به کار خود ادامه دهد $\rightarrow$ **بن‌بست رخ داده است.**

مثال ساده آموزشی

تصور کنید سه کاربر در یک کافی‌نت می‌خواهند بازی گروهی انجام دهند و هرکدام برای شروع نیاز به دو وسیله دارند:

- **نفر اول (T_1):** موس دارد و منتظر کیبورد است.
- **نفر دوم (T_2):** کیبورد و هدست دارد و منتظر موس است.
- **نفر سوم (T_3):** موس دارد و منتظر هدست است.

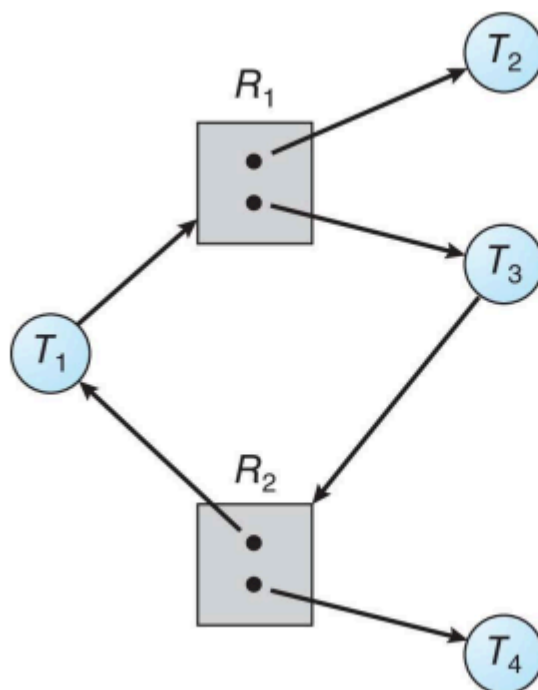
هرکس منتظر وسیله‌ای است که دیگری دارد، و هیچ‌کس نمی‌تواند بازی را شروع کند $\rightarrow$ **بن‌بست رخ داده است.**

نکات کلیدی

- **بن‌بست = وجود چرخه‌ی بسته از نخ‌ها که هرکدام منتظر منابعی هستند که توسط دیگری نگه داشته شده است.**
- نمودار تخصیص منابع ابزار مؤثری برای **شناسایی چرخه‌های انتظار** و تحلیل وضعیت‌های بن‌بست است.
- در صورتی که همه منابع فقط **یک نمونه** داشته باشند، وجود چرخه **قطعاً** به معنای وجود بن‌بست است.
- اگر منابع چند نمونه داشته باشند، چرخه **ممکن است یا ممکن است به بن‌بست منجر نشود.**

گرافی با چرخه اما بدون بن‌بست

Graph with a Cycle But no Deadlock



توضیح کامل از تصویر

این نمودار نشان می‌دهد که وجود چرخه در گراف تخصیص منابع، لزوماً به معنای وقوع بن‌بست نیست. در این مثال، با وجود تشکیل یک چرخه بین نخ‌ها و منابع، بن‌بست اتفاق نیفتاده زیرا هنوز امکان انجام پردازش و آزادسازی منابع وجود دارد.

اجزای نمودار

منابع:

- (• •) دارای دو نمونه: R_1
- (• •) دارای دو نمونه: R_2

نخ‌ها:

- T_1 و نگهداری R_1 در حال درخواست: T_1
- R_1 نگهدارنده یکی از نمونه‌های: T_2
- R_2 و نگهداری R_2 در حال درخواست: T_3
- R_2 نگهدارنده یکی از نمونه‌های: T_4

تحلیل دقیق

در این گراف، چرخه‌ای بین T_1 ، R_1 ، T_3 ، R_2 و T_1 تشکیل شده است:

$T_1 \rightarrow R_1 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$

اما با دقت در وضعیت منابع متوجه می‌شویم که:

- است T_3 و یکی نزد T_2 دو نمونه دارد، یکی نزد R_1 .
- است T_4 و یکی نزد T_1 نیز دو نمونه دارد، یکی نزد R_2 .

به دلیل وجود چندین نمونه از هر منبع، این امکان وجود دارد که:

- یکی از نخ‌ها (مثلاً T_2 یا T_4) وظیفه‌اش را تمام کرده و منبع را آزاد کند.
- سپس نخ‌ی که منتظر آن منبع است، بتواند آن را دریافت کرده و ادامه دهد.
- در نتیجه چرخه شکسته می‌شود و بن‌بست رخ نمی‌دهد.

✓ پس: چرخه $\neq$ بن‌بست، مگر اینکه همه منابع فقط یک نمونه داشته باشند.

مثال ساده آموزشی

فرض کنید در یک آزمایشگاه دو دستگاه چاپگر (R_1) و دو اسکنر (R_2) وجود دارد.

- دانشجوی ۱ (T_1): یک اسکنر در اختیار دارد و منتظر چاپگر است.
- دانشجوی ۲ (T_2): یک چاپگر در اختیار دارد.
- دانشجوی ۳ (T_3): یک چاپگر دارد و منتظر اسکنر است.
- دانشجوی ۴ (T_4): یک اسکنر در اختیار دارد.

در این حالت با اینکه یک چرخه بین کاربران و دستگاه‌ها شکل گرفته، اما چون از هر دستگاه دو عدد موجود است، کاربران می‌توانند پس از اتمام کار، دستگاه‌ها را آزاد کرده و دیگران از آن استفاده کنند $\rightarrow$ بن‌بست اتفاق نمی‌افتد.

نکات کلیدی

- وجود چرخه در گراف تخصیص منابع، شرط لازم برای بن بست است ولی کافی نیست.
- فقط در حالتی که همه منابع فقط یک نمونه داشته باشند، وجود چرخه معادل با بن بست است.
- اگر حداقل یک منبع دارای چند نمونه باشد، امکان شکستن چرخه و جلوگیری از بن بست وجود دارد.
- بررسی دقیق تعداد نمونه‌های منابع در تحلیل وضعیت بن بست بسیار مهم است.

حقایق پایه و روش‌های مدیریت بن بست

Basic Facts & Methods for Handling Deadlocks

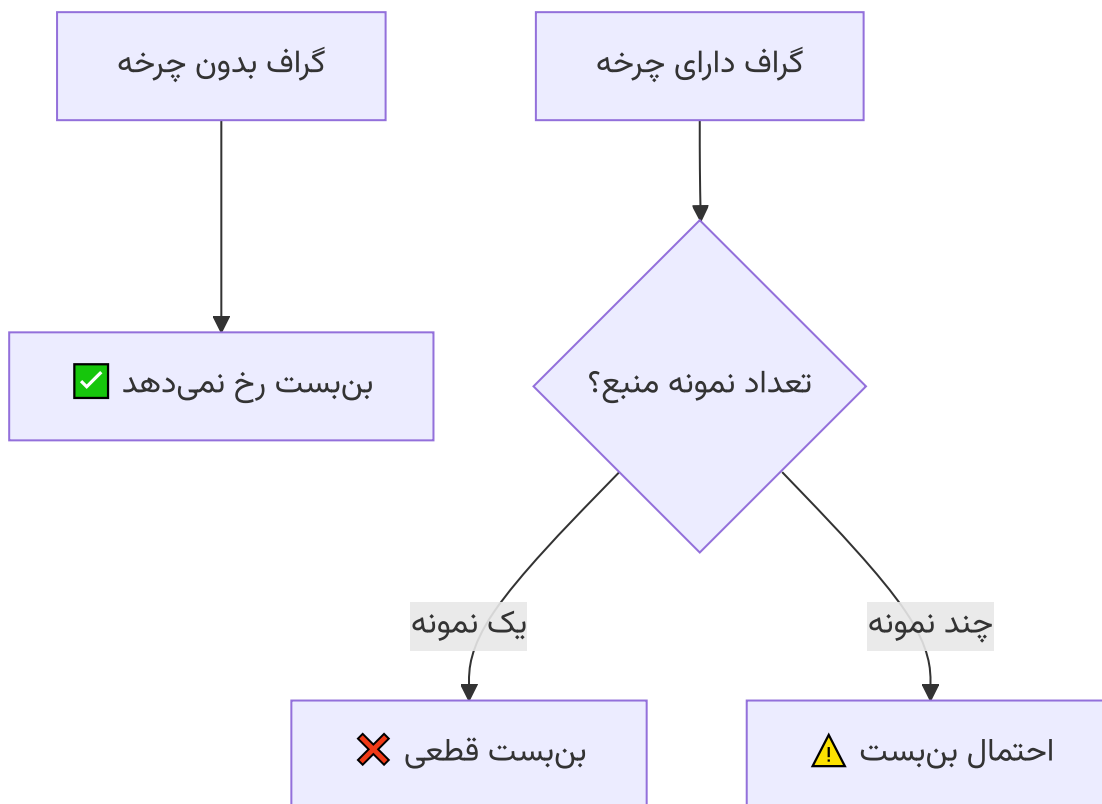
بخش اول: حقایق پایه درباره گراف تخصیص منابع

در سیستم‌هایی که از **گراف تخصیص منابع** (Resource Allocation Graph - RAG) استفاده می‌شود، می‌توان وضعیت بن بست را با بررسی وجود چرخه تحلیل کرد.

قوانین مهم:

- اگر گراف چرخه نداشته باشد → بن بست وجود ندارد.
- اگر گراف دارای چرخه باشد → دو حالت دارد:
 - اگر از هر نوع منبع فقط یک نمونه وجود داشته باشد → بن بست قطعی است.
 - اگر از برخی منابع چند نمونه وجود داشته باشد → امکان بن بست وجود دارد، اما قطعی نیست.

خلاصه تصویری:



بخش دوم: روش‌های برخورد با بن بست

در طراحی سیستم‌عامل‌ها، چهار رویکرد کلی برای مقابله با بن بست وجود دارد:

1. پیشگیری از بن بست (Deadlock Prevention)

با طراحی سیستم به گونه ای که یکی از شرایط لازم برای بن بست هرگز برقرار نشود. مثال: عدم اجازه نگهداری همزمان چند منبع → حذف شرط "Hold and Wait".

2. اجتناب از بن بست (Deadlock Avoidance)

با بررسی وضعیت سیستم قبل از اعطای منابع، تصمیم گیری می شود که این تخصیص منجر به بن بست خواهد شد یا نه. ابزار مهم: الگوریتم بانکدار (Banker's Algorithm).

3. تشخیص و بازیابی از بن بست (Detection and Recovery)

در این روش، سیستم اجازه می دهد که بن بست رخ دهد و سپس:

- آن را تشخیص می دهد،
- و اقدام به بازیابی می کند (مانند آزاد کردن منابع یا خاتمه پردازش).

4. نادیده گرفتن بن بست (Ignore Deadlocks)

رویکرد رایج در سیستم های ساده مانند بسیاری از نسخه های ویندوز یا لینوکس:

"بگذار اتفاق بیفتد، نگرانش نباشیم!"

پیشگیری از بن بست (Deadlock Prevention)

مقدمه

در پیشگیری از بن بست، هدف این است که سیستم را به گونه ای طراحی کنیم که حداقل یکی از چهار شرط لازم برای وقوع بن بست هرگز برقرار نشود.

چهار شرط وقوع بن بست:

- محرومیت متقابل (Mutual Exclusion)
- نگهداری و انتظار (Hold and Wait)
- عدم امکان پیش دستی (No Preemption)
- انتظار چرخشی (Circular Wait)

1. محرومیت متقابل (Mutual Exclusion)

- تنها در منابع غیرقابل اشتراک باید برقرار باشد. مثلاً:
- فایل های فقط خواندنی می توانند توسط چند فرآیند استفاده شوند → نیاز به محرومیت متقابل ندارند.
- برای منابعی مثل پرینتر یا CPU که فقط یک استفاده کننده دارند، این شرط اجتناب ناپذیر است.

2. نگهداری و انتظار (Hold and Wait)

برای جلوگیری از این شرط:

- فرآیند نباید هیچ منبعی در اختیار داشته باشد زمانی که درخواست منبع جدیدی می دهد.
- دو راهکار:
- 1. فرآیند باید همه منابع مورد نیازش را یکجا درخواست دهد و اگر همه مهیا شد، اجرا را شروع کند.
- 2. فرآیند تنها زمانی می تواند منابع جدید درخواست کند که هیچ منبعی در اختیار نداشته باشد.

● مشکلات این روش:

- استفاده ضعیف از منابع (Resource Underutilization)
- احتمال گرسنگی فرآیندها (Starvation)

۳. عدم امکان پیش‌دستی (No Preemption)

جلوگیری از این شرط با سیاست زیر:

- اگر فرآیندی منابعی دارد و منبع جدیدی را درخواست کند که در دسترس نیست:
- تمام منابعی که در اختیار دارد، از او گرفته می‌شود.
- این منابع به لیست انتظار اضافه می‌شوند.
- فرآیند تنها زمانی مجاز به اجرا است که همه منابع قدیمی و جدید برایش فراهم شوند.

۴. انتظار چرخشی (Circular Wait)

برای حذف این شرط:

- یک ترتیب کلی (Total Ordering) برای منابع تعریف می‌کنیم (مثلاً $R1 < R2 < R3 < \dots$).
- فرآیندها باید منابع را فقط به ترتیب افزایشی درخواست کنند.

مثلاً:

- اگر فرآیندی $R2$ را در اختیار دارد، فقط مجاز به درخواست $R3$ یا $R4$ است، نه $R1$.

مثال ساده آموزشی

فرض کنید در یک آزمایشگاه، تنها یک میکروسکوپ (منبع غیرقابل اشتراک) وجود دارد. سه دانشجو به ترتیب منابع مورد نیاز خود را درخواست می‌کنند.

اگر:

- هر دانشجو فقط زمانی درخواست بدهد که هیچ منبعی در اختیار ندارد
- یا منابع را بر اساس شماره‌گذاری مشخصی (مثلاً میکروسکوپ > سانتریفیوژ > یخچال) درخواست دهد

→ بن‌بست رخ نخواهد داد.

نکات کلیدی

- برای پیشگیری از بن‌بست باید حداقل یکی از شروط چهارگانه را حذف کنیم.
- حذف شرط "Hold and Wait" و "Circular Wait" بیشتر مرسوم است.
- پیشگیری همیشه موفق است، اما ممکن است منجر به کاهش بهره‌وری سیستم شود.
- پیشگیری ≠ اجتناب؛ پیشگیری با طراحی اولیه انجام می‌شود، اجتناب به تحلیل لحظه‌ای نیاز دارد.

Circular Wait – پیشگیری از شرط انتظار چرخشی

مفهوم اصلی:

هر کدام منابعی را نگه دارند و در عین حال منتظر منابعی، (threads) زمانی رخ می‌دهد که مجموعه‌ای از فرآیندها یا نخ‌ها Circular Wait باشند که توسط نخ بعدی در زنجیره نگه داشته شده است.

برای پیشگیری:

- به هر منبع (مثل mutex) یک شماره یکتا اختصاص می‌دهیم.
- تمام منابع باید به ترتیب افزایشی شماره‌شان تخصیص یابند.

شرایط مثال اسلاید:

```
first_mutex = 1;  
second_mutex = 5;
```

در این مثال:

- ترتیب صحیح گرفتن منابع باید از first_mutex به second_mutex باشد، چون $5 > 1$.
- بنابراین thread_two نباید منابع را به ترتیب 5 → 1 درخواست کند.

کد توضیحی اسلاید:

✓ کد صحیح برای thread_one :

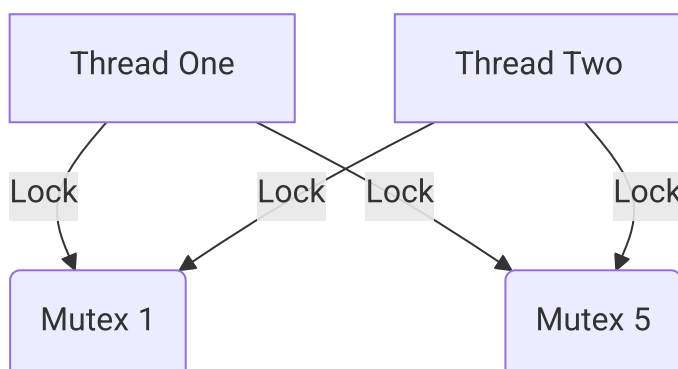
```
void *do_work_one(void *param) {  
    pthread_mutex_lock(&first_mutex);    // شماره 1  
    pthread_mutex_lock(&second_mutex);   // شماره 5  
    /* کار انجام شود */  
    pthread_mutex_unlock(&second_mutex);  
    pthread_mutex_unlock(&first_mutex);  
    pthread_exit(0);  
}
```

✗ کد نادرست برای thread_two :

```
void *do_work_two(void *param) {  
    pthread_mutex_lock(&second_mutex);   // شماره 5  
    pthread_mutex_lock(&first_mutex);    // شماره 1 ← خطا! ترتیب کاهش یافته  
    /* کار انجام شود */  
    pthread_mutex_unlock(&first_mutex);  
    pthread_mutex_unlock(&second_mutex);  
    pthread_exit(0);  
}
```

این کد باعث نقض شرط ترتیب منابع و در نتیجه احتمال وقوع Circular Wait می‌شود.

Mermaid: نمونه‌ای از ترتیب مجاز و غیرمجاز



✓ مسیر A ترتیب صعودی دارد → مجاز
✗ مسیر B ترتیب نزولی دارد → ممکن است بن‌بست ایجاد کند

مثال ساده آموزشی:

فرض کنید منابع شماره‌گذاری شده‌اند:

- Mutex A: 2
- Mutex B: 4

در این صورت، نخ‌ها باید فقط این ترتیب را رعایت کنند:

- ابتدا گرفتن A و سپس B مجاز است.
- گرفتن B و سپس A مجاز نیست.

نکات کلیدی

- جلوگیری از Circular Wait با ترتیب عددی منابع، ساده و بسیار مؤثر است.
- هر نخ باید منابع را فقط به ترتیب افزایشی شماره درخواست کند.
- این روش در سیستم‌های چندنخی (Multithreading) با mutexها بسیار کاربردی است.
- اجرای نادرست این ترتیب ممکن است باعث بن‌بست شود.

Deadlock Avoidance – اجتناب از بن‌بست

تعریف کلی

اجتناب از بن‌بست به معنای آن است که سیستم، پیش از تخصیص منابع به هر فرآیند یا نخ (Thread)، بررسی می‌کند که آیا این تخصیص باعث می‌شود سیستم در آینده وارد وضعیت بن‌بست شود یا خیر. اگر احتمال بن‌بست وجود داشته باشد، تخصیص انجام نمی‌شود.

پیش‌نیاز اصلی الگوریتم‌های اجتناب از بن‌بست

برای اجرای این رویکرد، سیستم نیاز دارد که هر نخ (Thread) از قبل اعلام کند:

"حداکثر چه تعداد از هر نوع منبع را ممکن است در طول اجرای خود نیاز داشته باشم؟"

این اطلاعات به سیستم کمک می‌کند وضعیت آینده را مدل کند و تشخیص دهد که تخصیص یک منبع خاص امن (Safe) هست یا نه.

وضعیت تخصیص منابع (Resource-Allocation State)

برای تصمیم‌گیری در مورد تخصیص منابع، سیستم از یک مدل استفاده می‌کند که شامل سه عنصر کلیدی است:

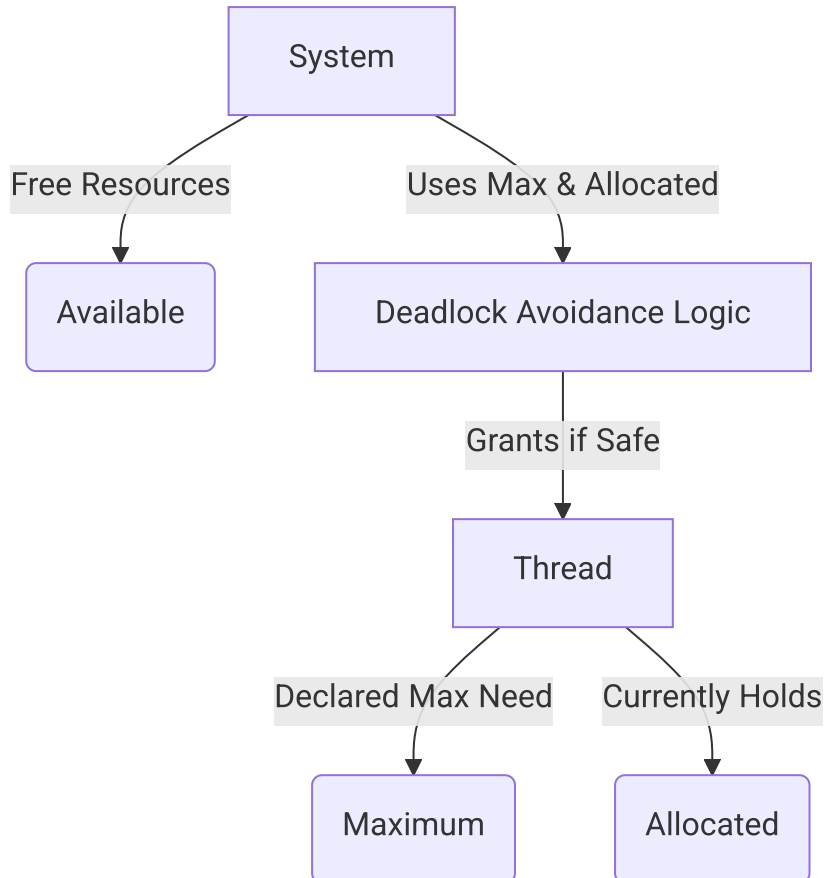
- تعداد منابع موجود (Available): تعداد منابع آزاد فعلی از هر نوع.
 - تخصیص‌یافته‌ها (Allocated): تعداد منابعی که هم‌اکنون به هر نخ یا فرآیند تخصیص یافته است.
 - حداکثر نیاز (Maximum Demand): بیشترین تعداد منابعی که هر نخ ممکن است در آینده درخواست کند.
- با این سه مؤلفه، سیستم می‌تواند شبیه‌سازی کند که آیا تخصیص بعدی منجر به بن‌بست خواهد شد یا خیر.

الگوریتم کلاسیک: الگوریتم بانکدار (Banker's Algorithm)

الگوریتم مشهور در این حوزه "Banker's Algorithm" نام دارد که توسط Dijkstra معرفی شده است.

- مانند یک بانکدار فرضی عمل می‌کند که فقط زمانی به مشتری وام می‌دهد که مطمئن باشد حتی بعد از تخصیص وام، بانک توانایی پاسخگویی به نیاز سایر مشتری‌ها را نیز دارد.
- اگر تخصیص جدید باعث شود سیستم وارد وضعیت ناامن (Unsafe) شود، درخواست رد می‌شود.

دیاگرام Mermaid: اجزای وضعیت تخصیص منابع



مثال ساده آموزشی

فرض کنید:

- نیاز داشته باشد X ممکن است حداکثر به 10 واحد از منبع Thread A.
- فعلاً 5 واحد دارد.
- سیستم فقط 3 واحد آزاد از X دارد.

اگر Thread A درخواست 3 واحد دیگر دهد، سیستم بررسی می‌کند:

- آیا بعد از تخصیص این 3 واحد، وضعیت باقی‌مانده سیستم اجازه می‌دهد سایر فرآیندها هم به نیازشان برسند؟

اگر پاسخ **بله** باشد → تخصیص انجام می‌شود.

اگر پاسخ **نه** باشد → تخصیص رد می‌شود تا از بن‌بست جلوگیری شود.

نکات کلیدی

- در اجتناب از بن‌بست، شرط‌های بن‌بست حذف نمی‌شوند؛ بلکه سیستم با تحلیل پویا، از ورود به وضعیت بن‌بست جلوگیری می‌کند.
- نیازمند دانستن **حداکثر نیاز هر فرآیند** در زمان آغاز آن است.
- **وضعیت تخصیص منابع** شامل: منابع آزاد، منابع تخصیص‌یافته، و حداکثر نیاز.
- از این اطلاعات برای تصمیم‌گیری امن استفاده می‌کند Banker's Algorithm.
- در سیستم‌هایی با منابع محدود و بحرانی، اجتناب از بن‌بست روش بسیار مناسبی است.

وضعیت ایمن در اجتناب از بن‌بست – Safe State

در این اسلاید، مفهوم **وضعیت ایمن (Safe State)** به‌عنوان یک ابزار کلیدی در اجتناب از بن‌بست معرفی شده است. هدف این است که قبل از تخصیص یک منبع، سیستم بررسی کند که آیا این تخصیص منجر به ورود به وضعیت **ناایمن (Unsafe)** و در نهایت **بن‌بست** خواهد شد یا نه.

تعریف وضعیت ایمن (Safe State)

سیستم در **وضعیت ایمن** است اگر بتوان ترتیب مشخصی از اجرای تمام نخ‌ها پیدا کرد که در آن:

- هر نخ بتواند تمام منابع مورد نیازش را در زمانی مناسب دریافت کند.
 - پس از اجرای کامل، منابعش را آزاد کرده و به نخ بعدی در ترتیب، امکان تخصیص بدهد.
- ✓ اگر چنین ترتیبی وجود داشته باشد، سیستم در وضعیت **Safe** است.
- ✗ اگر هیچ ترتیبی یافت نشود، سیستم وارد وضعیت **Unsafe** شده که ممکن است به **بن‌بست** ختم شود.

توضیح ساده‌تر

فرض کنید نخ‌هایی داریم:

$\langle T1, T2, \dots, Tn \rangle$

اگر:

- نخ $T1$ بتواند با منابع موجود اجرا شود،
 - پس از اتمام، منابعش را آزاد کند و به $T2$ اجازه اجرا دهد،
 - و همین روند تا Tn ادامه یابد،
- آنگاه سیستم در وضعیت ایمن قرار دارد.

شرایط لازم برای Safe State

برای هر نخ T_i در ترتیب فرضی:

- اگر منابع مورد نیاز T_i در حال حاضر موجود نیست، باید بتواند **منتظر بماند** تا نخ‌های T_j با $i < j$ اجرا شده و منابعشان را آزاد کنند.
- پس از آن، T_i منابع را دریافت می‌کند، اجرا می‌شود و منابع را پس می‌دهد.

مثال ساده آموزشی

فرض کنید:

- منابع موجود: 5 واحد
 - سه نخ داریم: A، B و C
 - نیازهای کلی آن‌ها:
- نیاز کل = 3، تخصیص فعلی = 1: A
 - نیاز کل = 4، تخصیص فعلی = 2: B

- نیاز کل = 2، تخصیص فعلی = 1: C

منابع آزاد = $5 - (1+2+1) = 1$

الگوریتم بررسی می‌کند که:

- آیا می‌توان یک ترتیب برای اجرای A، B، C پیدا کرد که در آن، هر نخ بتواند به منابع مورد نیاز خود برسد؟
- مثلاً اگر اول A اجرا شود، منابع آزاد می‌کند → B اجرا شود → سپس C → وضعیت ایمن است. ✓

نکات کلیدی

- وضعیت ایمن به معنی **عدم وجود بن‌بست نیست**، بلکه به این معنی است که **با تخصیص فعلی می‌توان از بن‌بست جلوگیری کرد**.
- سیستم باید **قبل از تخصیص منابع** بررسی کند که آیا وضعیت ایمن باقی می‌ماند.
- اگر تخصیص باعث Unsafe State شود، باید رد شود یا به تعویق بیفتد.
- بررسی وضعیت ایمن، پایه و اساس **الگوریتم Banker's Algorithm** است.

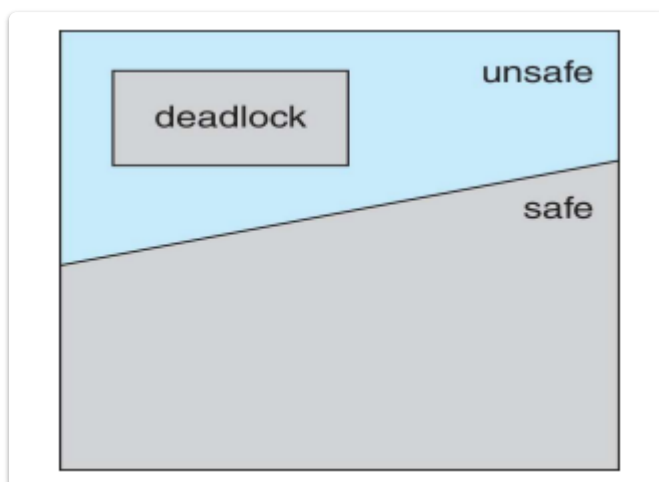
حقایق پایه درباره وضعیت ایمن و نایمن – Basic Facts

این اسلاید نکات پایه‌ای و بسیار مهمی را درباره رابطه بین **وضعیت ایمن (Safe State)**، **وضعیت نایمن (Unsafe State)** و **بن‌بست (Deadlock)** بیان می‌کند.

حقایق کلیدی

- ✓ اگر سیستم در وضعیت ایمن باشد
⇒ هرگز وارد بن‌بست نخواهد شد.
- ⚠ اگر سیستم در وضعیت نایمن باشد
⇒ امکان ورود به بن‌بست وجود دارد (ولی حتمی نیست).
- 🔒 هدف از اجتناب از بن‌بست (Deadlock Avoidance)
⇒ اطمینان از این است که سیستم هرگز وارد وضعیت نایمن نشود.

تحلیل تصویر (نمودار)



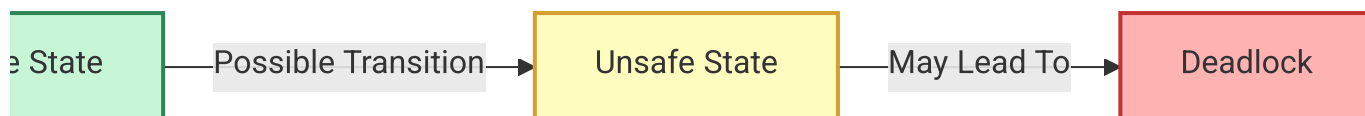
- فضای مستطیلی، کل **وضعیت‌های ممکن سیستم** را نشان می‌دهد.
- ناحیه‌ی خاکستری روشن: وضعیت‌های **ایمن (safe)**.
- ناحیه‌ی سفید بالایی: وضعیت‌های **نایمن (unsafe)**.
- ناحیه‌ی تیره درون unsafe: وضعیت‌هایی که منجر به **بن‌بست (deadlock)** می‌شوند.



این مدل نشان می‌دهد که:

- ورود به منطقه‌ی unsafe، ممکن است به منطقه‌ی deadlock ختم شود.
- اما تا زمانی که سیستم در منطقه‌ی safe باقی بماند، بن‌بست غیرممکن است.

دیagram Mermaid مفهومی



مثال ساده آموزشی

فرض کنید سه نخ داریم: A، B، C
و سیستم در وضعیت زیر است:

- نیاز دارد به 2 واحد دیگر از یک منبع A.
- فقط 1 واحد نیاز دارد و می‌تواند اجرا شود B.
- اگر B اجرا شود و منابع را آزاد کند، A و سپس C هم می‌توانند اجرا شوند.

بنابراین:

- اگر سیستم ابتدا منابع را به B بدهد، وضعیت ایمن باقی می‌ماند.
- اگر به جای B، منابع به A تخصیص داده شود، ممکن است هیچ نخ دیگری نتواند ادامه دهد و سیستم به بن‌بست برسد ⇒ نایمن.

نکات کلیدی

- وضعیت ایمن یعنی وجود یک ترتیب اجرا که از بن‌بست جلوگیری می‌کند.
- وضعیت نایمن لزوماً بن‌بست نیست، اما امکان رسیدن به آن وجود دارد.
- اجتناب از بن‌بست تضمین می‌کند که سیستم هرگز وارد وضعیت نایمن نشود.
- تصویر این اسلاید، مرز مفهومی بین safe، unsafe و deadlock را به خوبی نمایش می‌دهد.

Deadlock Avoidance Algorithms – الگوریتم‌های اجتناب از بن‌بست

در این اسلاید، دو الگوریتم اصلی برای اجتناب از بن‌بست بسته به نوع منابع معرفی شده‌اند. نوع منابع در سیستم‌ها به دو دسته تقسیم می‌شود:

1. منابع با تنها یک نمونه (Single Instance)

📌 برای منابعی که تنها یک نمونه از آن‌ها در سیستم وجود دارد (مثل چاپگر یا دیسک خاص)، از نمودار تخصیص منابع (Resource Allocation Graph – RAG) استفاده می‌شود.

🌟 نحوه کار:

نمودار تخصیص منابع گرافی است شامل:

- گره‌های فرآیندها (Ti)
- گره‌های منابع (Rj)

- یال‌هایی برای نمایش وضعیت درخواست، تخصیص یا ادعا

انواع یال‌ها:

- **Claim Edge ($T_i \rightarrow R_j$):**
نشان می‌دهد که نخ T_i ممکن است در آینده منبع R_j را درخواست کند.
✎ با خط چین‌دار نمایش داده می‌شود.
- **Request Edge ($T_i \rightarrow R_j$):**
وقتی نخ T_i واقعاً منبع R_j را درخواست کند، claim edge به request edge تبدیل می‌شود.
✎ خط پیوسته یک‌طرفه از نخ به منبع.
- **Assignment Edge ($R_j \rightarrow T_i$):**
وقتی R_j به T_i تخصیص داده شود، request edge به assignment edge تبدیل می‌شود.
✎ خط پیوسته یک‌طرفه از منبع به نخ.
□ وقتی منبع آزاد می‌شود، assignment edge به claim edge بازمی‌گردد.

شرط مهم:

نخ‌ها باید از قبل (*a priori*) اعلام کنند که ممکن است در آینده چه منابعی را نیاز داشته باشند.

2. منابع با چند نمونه (Multiple Instances)

🔴 در حالتی که از هر منبع چند نمونه در دسترس است (مثلاً 3 چاپگر یا 5 کانال ارتباطی)، از الگوریتم بانکدار (Banker's Algorithm) استفاده می‌شود.

در این اسلاید، وارد جزئیات الگوریتم بانکدار نشده، اما در اسلایدهای قبلی اشاره شد که این الگوریتم بر اساس:

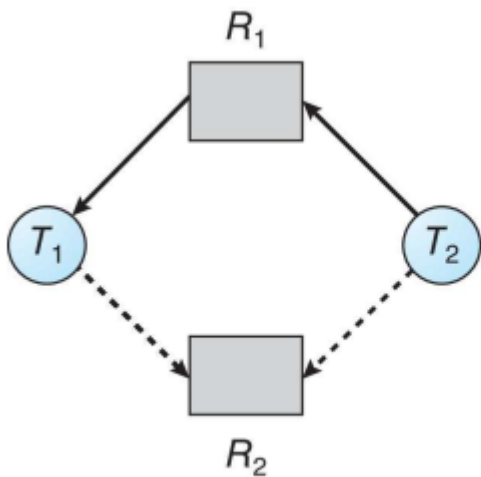
- بیشترین نیاز اعلام‌شده،
- تخصیص فعلی،
- و منابع آزاد سیستم،

تشخیص می‌دهد که آیا تخصیص بعدی باعث Unsafe State خواهد شد یا نه.

نکات کلیدی

- الگوریتم مناسب بستگی به نوع منابع دارد:
 - منابع با یک نمونه → نمودار تخصیص منابع (RAG)
 - منابع با چند نمونه → الگوریتم بانکدار
- در RAG:
 - Claim Edge → Request Edge → Assignment Edge → Claim Edge
 - سیستم باید قبل از تخصیص، با استفاده از این گراف بررسی کند که تخصیص جدید باعث ایجاد بن‌بست نمی‌شود.
 - کلید اصلی: اعلام نیازهای بالقوه توسط نخ‌ها قبل از اجرا (Claiming resources a priori)

نمودار تخصیص منابع – Resource-Allocation Graph



اجزای نمودار:

گره‌ها:

- دایره‌ها: نخ‌ها یا فرآیندها (در اینجا: T_1 و T_2)
- مربع‌ها: منابع (در اینجا: R_1 و R_2)

پال‌ها:

-  پال‌های پیوسته:

- $R_1 \rightarrow T_1$ و $R_1 \rightarrow T_2$: یعنی R_1 در حال حاضر به T_1 و T_2 **تخصیص داده شده‌اند**.

-  پال‌های چین‌دار (dashed):

- $T_1 \rightarrow R_2$ و $T_2 \rightarrow R_2$: یعنی T_1 و T_2 ممکن است در آینده R_2 را **درخواست کنند** (Claim Edges).

تحلیل وضعیت فعلی گراف

در حال حاضر:

- استفاده می‌کنند (یا درخواست داده‌اند و تخصیص انجام شده) R_1 هر کدام از T_1 و T_2 .
- هر دو نخ **ادعا کرده‌اند** که در آینده به R_2 نیاز خواهند داشت.

نکته مهم:

- هنوز هیچ Request واقعی برای R_2 ثبت نشده است.
- فقط ادعا (Claim) شده که شاید در آینده درخواست دهند.

مثال آموزشی

فرض کنید:

- قرار دارد (مثلاً دو نمونه دارد) T_1 و T_2 یک چاپگر است، که در حال حاضر هر دو نخ R_1 .
- یک اسکنر است، که فعلاً آزاد است ولی هنوز کسی آن را درخواست نکرده R_2 .
- اما هر دو نخ در آینده **ممکن است** R_2 را درخواست کنند (Claim Edge).

 سیستم باید بررسی کند که اگر یکی از نخ‌ها درخواست واقعی برای R_2 بدهد، آیا این تخصیص باعث ورود سیستم به وضعیت نالایم یا بن‌بست خواهد شد یا خیر.

- نشان‌دهنده‌ی نیازهای بالقوه در آینده است **Claim Edge**.
- با تبدیل claim edge به request edge، سیستم باید بررسی کند که آیا تخصیص جدید باعث ایجاد **چرخه** می‌شود یا خیر.
- در صورتی که **چرخه‌ای در گراف ایجاد شود** و فقط یک نمونه از هر منبع وجود داشته باشد $\Rightarrow$ **بن‌بست رخ داده است**.
- بنابراین، با استفاده از این گراف و بررسی وضعیت قبل از تخصیص، سیستم می‌تواند از ورود به بن‌بست جلوگیری کند.

وضعیت نایمن در نمودار – Unsafe State in Resource-Allocation Graph

تخصیص منابع

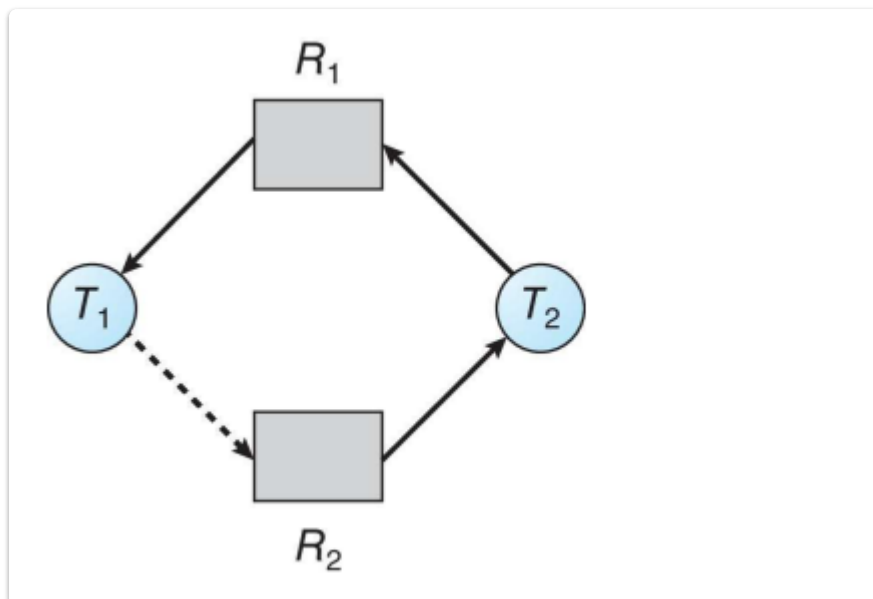
در این بخش، مفهوم "**Unsafe State**" (**وضعیت نایمن**) در قالب نمودار تخصیص منابع (Resource Allocation Graph – RAG) بررسی می‌شود. این حالت می‌تواند به **بن‌بست (Deadlock)** منجر شود اما الزاماً هنوز بن‌بست رخ نداده است.

✅ Safe State vs ❌ Unsafe State

وضعیت	تعریف	نتیجه
Safe State	وضعیت سیستم به گونه‌ای است که می‌توان ترتیب امنی از اجرای نخ‌ها یافت که هرکدام به منابعشان دسترسی پیدا کنند و آزاد کنند.	هیچ بن‌بستی اتفاق نخواهد افتاد 🛡️
Unsafe State	وضعیت سیستم ممکن است منجر به بن‌بست شود، چون تضمینی برای ترتیب امن وجود ندارد.	امکان وقوع بن‌بست ⚠️

Unsafe State in Resource-Allocation Graph

وضعیت نایمن در نمودار تخصیص منابع (RAG)



این اسلاید نمونه‌ای از **وضعیت نایمن (Unsafe State)** را در یک سیستم با استفاده از نمودار تخصیص منابع نشان می‌دهد. این حالت نشان می‌دهد که سیستم **ممکن است** به وضعیت بن‌بست (Deadlock) وارد شود، اما هنوز وارد آن نشده است.

🌿 توصیف گراف موجود

در گراف اسلاید داریم:

- دو فرآیند یا نخ (Thread): $T1$ و $T2$
- دو منبع (Resource): $R1$ و $R2$
- خطوط جهت‌دار:
- **پیکان از $R1$ به $T1$** : نشان می‌دهد که $R1$ در حال حاضر به $T1$ تخصیص داده شده است.
- **پیکان از $R2$ به $T2$** : نشان می‌دهد که $R2$ به $T2$ تخصیص داده شده است.
- **خط‌چین از $T1$ به $R2$** : یعنی $T1$ ادعا کرده که در آینده ممکن است $R2$ را درخواست کند (claim edge).

تحلیل وضعیت

در این وضعیت:

- نیاز داشته باشد $R2$ استفاده می‌کند و در آینده ممکن است به $R1$ در حال حاضر از $T1$ استفاده می‌کند.
 - اگر $T1$ واقعاً $R2$ را درخواست کند و همزمان $T2$ هم بخواهد $R1$ را، آنگاه یک **چرخه** در گراف شکل می‌گیرد.
- این وضعیت **ناایمن** است چون اگر همه فرآیندها حداکثر منابع اعلام‌شده خود را بخواهند، سیستم نمی‌تواند به صورت تضمینی آن‌ها را پشتیبانی کند، و ممکن است بن‌بست رخ دهد.

مثال ساده

فرض کنید:

- $T1$ = چاپگر را دارد، اما به اسکنر هم نیاز دارد، $(R1)$ چاپگر را دارد
 - و ممکن است به چاپگر نیاز داشته باشد (در آینده)، $(R2)$ اسکنر را دارد = $T2$
- اگر هر دو همزمان درخواست خود را بدهند، بن‌بست رخ می‌دهد.

نکات کلیدی

- نشان‌دهنده درخواست احتمالی آینده است (خط‌چین) Claim Edge.
- وضعیت ناایمن، خطری بالقوه برای بن‌بست است.
- اگر گراف شامل چرخه شود و منابع فقط یک نمونه داشته باشند، بن‌بست قطعی خواهد بود.
- الگوریتم‌هایی مانند Banker's Algorithm از ورود به این وضعیت جلوگیری می‌کنند.

Resource-Allocation Graph Algorithm

الگوریتم نمودار تخصیص منابع

این بخش به یکی از روش‌های **جلوگیری از بن‌بست (Deadlock Avoidance)** می‌پردازد که بر اساس **نمودار تخصیص منابع (Resource-Allocation Graph)** کار می‌کند. این روش مخصوص حالتی است که **هر نوع منبع فقط یک نمونه دارد**.

ایده‌ی اصلی الگوریتم

فرض کنید **نخی (Thread)** به نام T_i می‌خواهد **منبعی به نام R_j** را درخواست کند. سیستم باید بررسی کند که آیا این درخواست باعث ایجاد **چرخه در گراف تخصیص منابع می‌شود یا خیر**.

- اگر تبدیل لبه‌ی درخواست (Request Edge) به لبه‌ی تخصیص (Assignment Edge) باعث ایجاد چرخه شود → **درخواست رد می‌شود** چون ممکن است سیستم وارد وضعیت بن‌بست شود.
- اگر چرخه‌ای ایجاد نشود → **درخواست پذیرفته می‌شود** و منبع به نخ تخصیص داده می‌شود.

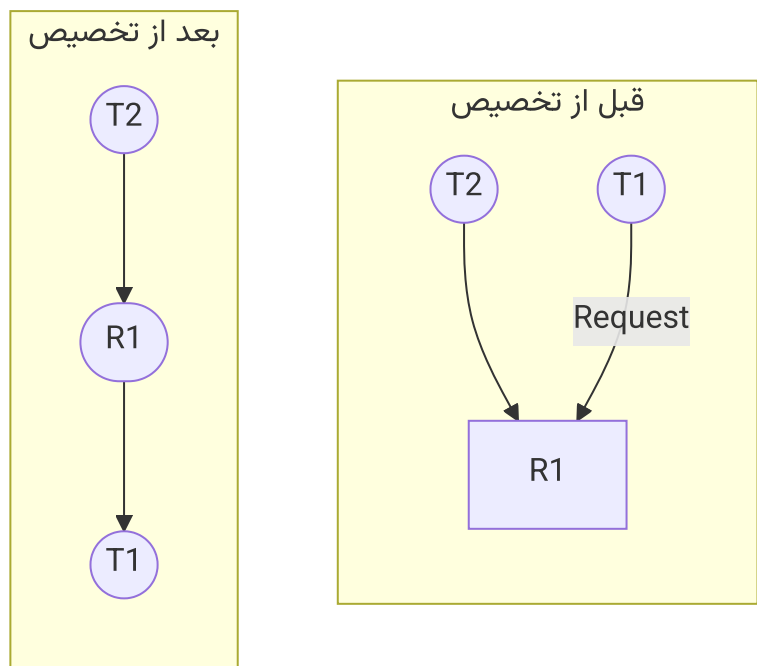
مراحل الگوریتم 🧠

1. وقتی نخ T_i منبع R_j را درخواست می‌کند:
 - یک **لبه‌ی درخواست** ($T_i \rightarrow R_j$) به گراف اضافه می‌شود.
2. بررسی می‌شود که آیا تبدیل این لبه به لبه‌ی تخصیص ($R_j \rightarrow T_i$) موجب ایجاد **چرخه در گراف** می‌شود یا نه.
3. اگر چرخه ایجاد نشد، منبع تخصیص داده می‌شود.
4. اگر چرخه ایجاد شد، سیستم درخواست را به حالت **منتظر (blocked)** می‌برد.

نمودار Mermaid – تبدیل Request به Assignment 📊

فرض کنید:

- در درخواست کرده و گراف فعلی شامل چرخه نمی‌شود $R1$ منبع $T1$
- بعد از تخصیص، اگر چرخه‌ای ایجاد نشود، تخصیص انجام می‌شود.



مثال ساده ✓

فرض کنیم:

- نیاز دارد $R1$ به منبع $T1$.
- قرار دارد نیاز داشته باشد $T1$ ممکن است در آینده به منبعی که در اختیار $T2$ نیست، اما $T2$ فعلاً در اختیار $R1$.

اگر همزمان درخواست‌ها بررسی نشوند و تبدیل به چرخه شوند، بن‌بست رخ می‌دهد. الگوریتم گراف تخصیص منابع با تحلیل **دینامیکی گراف**، از این چرخه‌ها جلوگیری می‌کند.

نکات کلیدی 📌

- این روش فقط در حالتی قابل استفاده است که **از هر نوع منبع تنها یک نمونه وجود دارد**.
- این الگوریتم با بررسی **ایجاد چرخه** در گراف تخصیص منابع، مانع ورود سیستم به بن‌بست می‌شود.
- **گراف باید به‌روز باشد** و همیشه وضعیت جاری تخصیص و درخواست را نشان دهد.
- اگر سیستم چرخه‌ای را تشخیص دهد، **درخواست به تعویق می‌افتد یا رد می‌شود**.

🏦 Banker's Algorithm (الگوریتم بانکدار)

الگوریتم بانکدار یکی از مهم‌ترین روش‌های جلوگیری از بن‌بست (Deadlock Avoidance) است و مخصوص حالتی است که از هر نوع منبع چند نمونه موجود است (برخلاف روش گراف تخصیص منابع که فقط برای یک نمونه جواب می‌دهد).

🧠 ایده‌ی اصلی

نام این الگوریتم از سیستم وام‌دهی بانک الهام گرفته شده است. در بانک، وام‌دهنده (سیستم) فقط در صورتی به مشتری (فرآیند) وام (منبع) می‌دهد که مطمئن باشد در نهایت همه می‌توانند بدهی خود را پس بدهند.

در این الگوریتم، هر فرآیند (نخ):

- از قبل اعلام می‌کند حداکثر چه مقدار از هر منبع نیاز دارد.
- وقتی درخواست می‌دهد، سیستم بررسی می‌کند آیا تخصیص فعلی سیستم را به یک وضعیت ایمن (Safe State) می‌برد یا نه.
- اگر ایمن بود → منبع تخصیص داده می‌شود؛
اگر نه → درخواست منتظر می‌ماند.

📊 ساختمان داده‌های مورد استفاده

فرض کنیم:

- فرآیند (نخ) در سیستم داریم n .
- نوع منبع مختلف داریم m .

ساختمان داده‌ها به این صورت‌اند:

نام بردار/ماتریس	توضیح	نوع
Available[m]	برداری که نشان می‌دهد از هر نوع منبع چه تعداد در حال حاضر آزاد است.	بردار
Max[n][m]	ماتریسی که نشان می‌دهد هر فرآیند حداکثر چه مقدار از هر منبع ممکن است بخواهد.	ماتریس
Allocation[n][m]	ماتریسی که نشان می‌دهد هر فرآیند اکنون چه مقدار از هر منبع در اختیار دارد.	ماتریس
Need[n][m]	مقدار منابعی که هر فرآیند هنوز نیاز دارد تا کار خود را تمام کند: $Need[i][j] = Max[i][j] - Allocation[i][j]$	ماتریس

🔄 مراحل اجرای الگوریتم

1. وقتی فرآیند P_i درخواست $Request[i]$ از منابع می‌دهد:

- بررسی شود آیا $Request[i] \leq Need[i]$ → اگر نه، خطا است.
- بررسی شود آیا $Request[i] \leq Available$ → اگر نه، باید منتظر بماند.
- اگر شرایط بالا برقرار بود:
- به صورت موقت:

- $Available = Available - Request[i]$
- $Allocation[i] = Allocation[i] + Request[i]$
- $Need[i] = Need[i] - Request[i]$

- سپس با الگوریتم بررسی وضعیت ایمن، چک می‌شود آیا سیستم در وضعیت ایمن باقی مانده یا نه.
- اگر ایمن بود → تخصیص نهایی می‌شود.
- اگر نایمن بود → تخصیص لغو می‌شود، فرآیند باید منتظر بماند.

مثال ساده

فرض کن:

- 3 فرآیند (P0, P1, P2)
- 3 نوع منبع (A, B, C)

- Available = [3, 3, 2]

Process	Max	Allocation	Need = Max - Allocation
P0	7 5 3	0 1 0	7 4 3
P1	3 2 2	2 0 0	1 2 2
P2	9 0 2	3 0 2	6 0 0

اگر P1 درخواست [2, 0, 1] بدهد:

- بررسی می‌کنیم که آیا سیستم پس از تخصیص به وضعیت ایمن خواهد رفت یا خیر.

نکات کلیدی

- الگوریتم Banker فقط زمانی کار می‌کند که فرآیندها از قبل حداکثر نیاز خود را اعلام کرده باشند.
- مهم‌ترین هدف این الگوریتم: اطمینان از باقی ماندن در وضعیت ایمن.
- اگر تخصیص منابع باعث شود سیستم وارد وضعیت ناایمن شود → منبع تخصیص داده نمی‌شود.
- محاسبات آن ممکن است پیچیده باشد اما جلوی وقوع بن‌بست را می‌گیرد.

✓ Safety Algorithm (الگوریتم بررسی وضعیت ایمن)

الگوریتم "Safety" بخشی کلیدی از Banker's Algorithm است. هدف آن این است که بررسی کند آیا پس از یک تخصیص منابع، سیستم در وضعیت ایمن باقی می‌ماند یا نه.

ورودی‌ها

- Available[m]: تعداد منابع آزاد از هر نوع.
- Max[n][m]: حداکثر منابعی که هر فرآیند ممکن است بخواهد.
- Allocation[n][m]: منابعی که در حال حاضر به هر فرآیند تخصیص یافته.
- Need[n][m] = Max - Allocation: منابعی که هر فرآیند هنوز نیاز دارد.

داده‌های کمکی

- Work[m]: برداری که مقدار فعلی منابع موجود را نگه می‌دارد.
- Finish[n]: برداری بولی که نشان می‌دهد هر فرآیند می‌تواند با منابع موجود تمام شود یا نه.

مراحل الگوریتم

مرحله 1: مقداردهی اولیه

Work = Available

```
For all i: Finish[i] = false
```

مرحله 2: جستجوی فرآیندی قابل اجرا

```
طوری که i پیدا کن  
Finish[i] == false  
AND  
Need[i] <= Work
```

اگر پیدا نشد → برو به مرحله 4.

مرحله 3: فرض کن فرآیند i اجرا شده

```
Work = Work + Allocation[i]  
Finish[i] = true  
برگرد به مرحله 2
```

مرحله 4: بررسی وضعیت نهایی

اگر تمام `Finish[i] == true` باشند → سیستم در وضعیت ایمن (Safe) است.

اگر حتی یک `Finish[i] == false` باقی مانده باشد → سیستم در وضعیت نایمن (Unsafe) قرار دارد.

مثال ساده

فرض کنید:

```
Available = [3, 3, 2]  
Need[1] = [1, 2, 2]  
Allocation[1] = [2, 0, 0]
```

اگر `Need[1] <= Work` باشد:

```
Work = Work + Allocation[1] = [3,3,2] + [2,0,0] = [5,3,2]  
Finish[1] = true
```

حالا بررسی ادامه دارد تا ببینیم همه فرآیندها می‌توانند به ترتیب مشابه تمام شوند.

نکات کلیدی

- این الگوریتم تخصیص واقعی انجام نمی‌دهد، بلکه شبیه‌سازی می‌کند که اگر این تخصیص انجام شود، آیا سیستم ایمن باقی می‌ماند؟
- اگر حداقل یک ترتیب از اجرای فرآیندها پیدا شود که همه به منابع مورد نیازشان برسند → وضعیت ایمن است.
- در غیر این صورت → وضعیت نایمن و امکان وقوع بن‌بست وجود دارد.

این الگوریتم بخشی از **Banker's Algorithm** است که هنگام دریافت یک درخواست جدید از یک فرآیند اجرا می‌شود و بررسی می‌کند که آیا می‌توان منابع درخواستی را بدون خطر ورود به وضعیت نایمن (unsafe) به فرآیند داد یا نه.

تعریفها

- $Request[i]$: فرآیند P_i برداری از منابعی که فرآیند P_i می‌خواهد از نوع R_j .
- اگر $Request[i][j] = k \rightarrow P_i$ یعنی P_i خواستار k واحد از منبع R_j است.

مراحل الگوریتم

✿ مرحله 1: بررسی ادعای بیش‌ازحد

خطا (فرآیند بیش از ادعای قبلی خود درخواست کرده) $\rightarrow Request[i] > Need[i]$ اگر

❌ درخواست غیرمجاز است.

✿ مرحله 2: بررسی در دسترس بودن منابع

ادامه بده $\rightarrow Request[i] \leq Available$ اگر
(کافی نیست resources) در غیر این صورت $\rightarrow$ فرآیند باید صبر کند

✿ مرحله 3: تخصیص فرضی منابع (شبیه‌سازی تخصیص)

سیستم "وانمود می‌کند" که منابع را اختصاص داده:

```
Available = Available - Request[i]
Allocation[i] = Allocation[i] + Request[i]
Need[i] = Need[i] - Request[i]
```

✅ اگر سیستم اکنون در حالت ایمن باشد (با اجرای Safety Algorithm):

$\rightarrow$ منابع واقعاً به P_i اختصاص داده می‌شوند.

❌ اگر سیستم وارد وضعیت نایمن شود:

$\rightarrow$ تخصیص لغو می‌شود و وضعیت قبلی بازگردانده می‌شود:

```
Available = Available + Request[i]
Allocation[i] = Allocation[i] - Request[i]
Need[i] = Need[i] + Request[i]
```

مثال ساده

فرض کنید:

```
Available = [3, 3, 2]
Allocation[1] = [1, 0, 1]
Max[1] = [3, 2, 2]
Request[1] = [1, 2, 1]
```

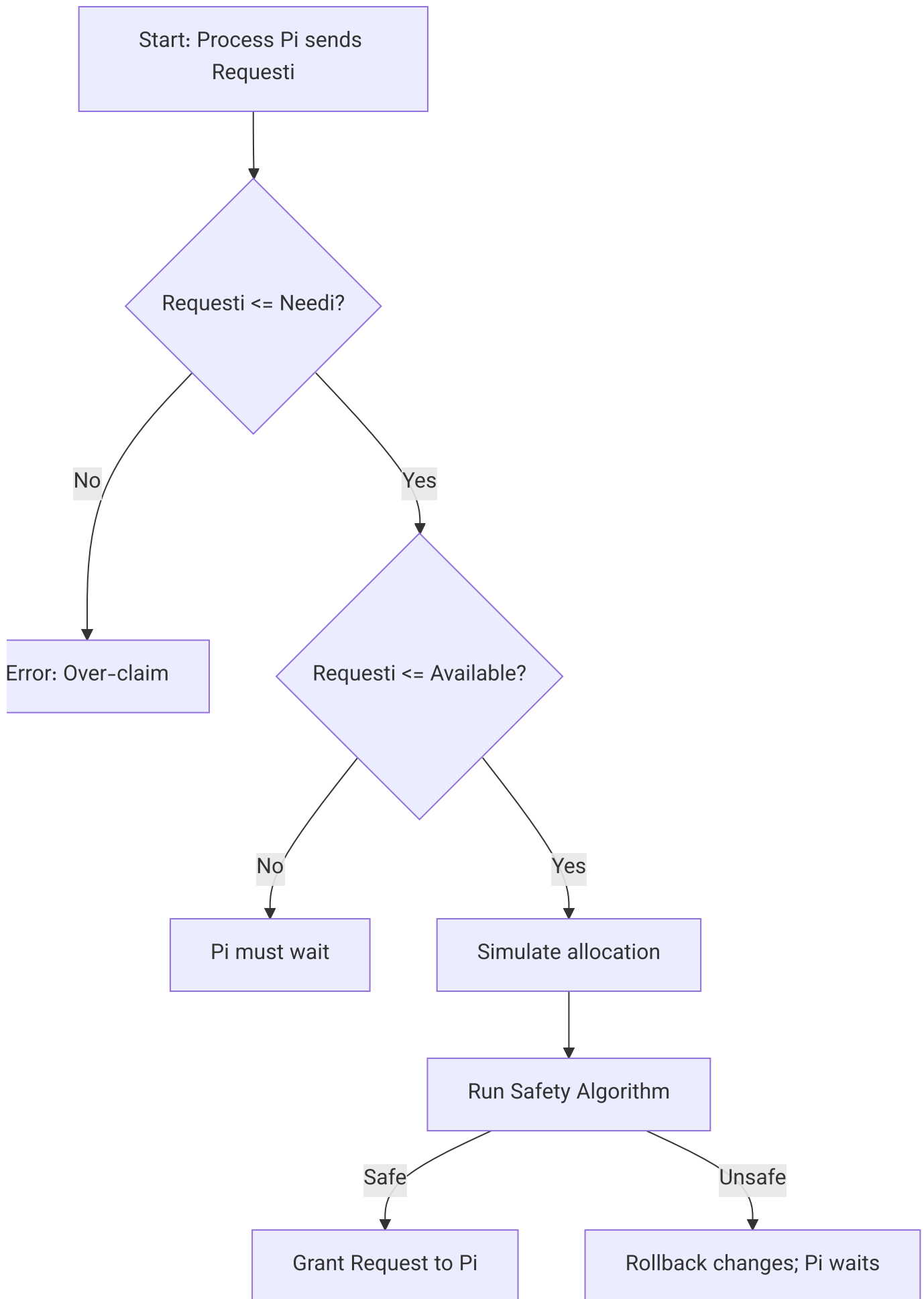
1. محاسبه $Need[1] = Max - Allocation = [2, 2, 1]$

2. بررسی: $Request[1] \leq Need[1] \rightarrow OK$

3. بررسی: $Request[1] \leq Available \rightarrow [1, 2, 1] \leq [3, 3, 2] \rightarrow OK$

حال تخصیص فرضی انجام می‌شود و بررسی می‌کنیم که سیستم **در وضعیت ایمن باقی می‌ماند یا نه** (با استفاده از الگوریتم "Safety").

 **دیاگرام ساده (جریان تصمیم)**



نکات کلیدی

- الگوریتم فقط زمانی تخصیص واقعی انجام می‌دهد که مطمئن شود سیستم وارد وضعیت نایمن نمی‌شود.
- مرحله سوم نوعی شبیه‌سازی تخصیص منابع است.
- اگر تخصیص فرضی منجر به وضعیت نایمن شود، سیستم به حالت قبل بازمی‌گردد.
- این الگوریتم به شدت به صحت ماتریس‌های Max ، Allocation و Available وابسته است.

مثال الگوریتم بانکدار (Banker's Algorithm)

این اسلاید یک نمونه از اجرای Banker's Algorithm را نشان می‌دهد. این الگوریتم برای پیشگیری از بن‌بست (Deadlock Avoidance) به کار می‌رود. ایده اصلی این است که سیستم تنها زمانی منابع را به یک پردازش می‌دهد که بعد از تخصیص، همچنان در یک حالت امن (Safe State) باقی بماند.

داده‌های مثال

- پردازش‌ها (Threads): ۵ پردازش T0 تا T4
- منابع (Resources): سه نوع

- تعداد کل 10 نمونه : A
- تعداد کل 5 نمونه : B
- تعداد کل 7 نمونه : C

جدول وضعیت در لحظه T0

پردازش	Allocation (A,B,C)	Max (A,B,C)
T0	0,1,0	7,5,3
T1	2,0,0	3,2,2
T2	3,0,2	9,0,2
T3	2,1,1	2,2,2
T4	0,0,2	4,3,3

- منابع آزاد (Available):

$$(A, B, C) = (3, 3, 2)$$

محاسبه ماتریس Need

فرمول:

$$\text{Need}[i] = \text{Max}[i] - \text{Allocation}[i]$$

پردازش	Need (A,B,C)
T0	(7,4,3)
T1	(1,2,2)
T2	(6,0,0)
T3	(0,1,1)
T4	(4,3,1)

اجرای Safety Algorithm

هدف: بررسی کنیم که ترتیب اجرای پردازها به گونه‌ای باشد که همه بتوانند کامل شوند.

• شروع: $Work = Available = (3, 3, 2)$

1. **T1**: $Need(1,2,2) \leq Work(3,3,2) \rightarrow$ قابل اجرا 

بعد از اجرا: $Work = Work + Allocation = (5, 3, 2)$

2. **T3**: $Need(0,1,1) \leq Work(5,3,2) \rightarrow$ قابل اجرا 

بعد از اجرا: $Work = (7, 4, 3)$

3. **T0**: $Need(7,4,3) \leq Work(7,4,3) \rightarrow$ قابل اجرا 

بعد از اجرا: $Work = (7, 5, 3)$

4. **T2**: $Need(6,0,0) \leq Work(7,5,3) \rightarrow$ قابل اجرا 

بعد از اجرا: $Work = (10, 5, 5)$

5. **T4**: $Need(4,3,1) \leq Work(10,5,5) \rightarrow$ قابل اجرا 

بعد از اجرا: $Work = (10, 5, 7)$

نتیجه

• سیستم در یک حالت امن قرار دارد.

• یک ترتیب امن برای اجرای پردازها وجود دارد:

$$\langle T_1, T_3, T_0, T_2, T_4 \rangle$$

نکات کلیدی

- تصمیم می‌گیرد کدام پرداز قابل اجراست **Need $\leq$ Work (Available)** با مقایسه‌ی Banker's Algorithm.
- اگر حداقل یک ترتیب امن برای اجرای همه پردازها وجود داشته باشد، سیستم **Safe State** دارد.
- مهم‌ترین محاسبه:

$$Need = Max - Allocation$$

• بروزسانی **Work** بعد از اتمام هر پرداز.

بررسی حالت امن (Safe Sequence)

با اجرای **Safety Algorithm**، سیستم تعیین می‌کند که آیا ترتیب امنی برای اجرای همه‌ی پردازها وجود دارد یا خیر.
یک دنباله امن (**Safe Sequence**) یافت شده است:

$$\langle T_1, T_3, T_4, T_2, T_0 \rangle$$

این به این معناست که پردازها می‌توانند به همین ترتیب اجرا شوند و پس از پایان هر کدام، منابع آزاد شده برای پردازهای بعدی کافی خواهد بود.

مثال توضیحی اجرای الگوریتم

فرض کنید منابع در ابتدا همانند قبل $(3, 3, 2)$ هستند:

1. **T1** است $\rightarrow$ اجرا می‌شود $Available(3,3,2) \leq$ نیاز $(1,2,2)$ دارد که **جدید** $Available(5,3,2) =$
2. **T3** است $\rightarrow$ اجرا می‌شود $Available(5,3,2) \leq$ نیاز $(0,1,1)$ دارد که **جدید** $Available(7,4,3) =$

3. نیاز (4,3,1) دارد که $(7,4,3) \geq$ است → اجرا می‌شود T4.
جدید Available (7,4,5) =
4. نیاز (6,0,0) دارد که $(7,4,5) \geq$ است → اجرا می‌شود T2.
جدید Available (10,4,7) =
5. نیاز (7,4,3) دارد که $(10,4,7) \geq$ است → اجرا می‌شود T0.
نهایی Available (10,5,7) =

🔑 جمع‌بندی نکات

- ماتریس Max - Allocation = **Need** و نشان‌دهنده‌ی منابع باقیمانده مورد نیاز هر پردازش است.
- اگر بتوان یک **ترتیب امن** پیدا کرد، سیستم در حالت Safe است.
- ترتیب امن در این مثال:

$$\langle T_1, T_3, T_4, T_2, T_0 \rangle$$

- حالت امن به این معناست که هیچ بن‌بستی رخ نخواهد داد.

توضیح اسلاید: Example of Banker's Algorithm

این اسلاید یک **نمونه از اجرای Banker's Algorithm** را نشان می‌دهد. این الگوریتم برای **پیشگیری از بن‌بست (Deadlock Avoidance)** به کار می‌رود. ایده اصلی این است که سیستم تنها زمانی منابع را به یک پردازش می‌دهد که بعد از تخصیص، همچنان در یک **حالت امن (Safe State)** باقی بماند.

◆ داده‌های مثال

- **پردازش‌ها (Processes):** ۵ پردازش (T0 تا T4)
- **منابع (Resources):** سه نوع
- تعداد کل ۱۰ نمونه: A
- تعداد کل ۵ نمونه: B
- تعداد کل ۷ نمونه: C

جدول وضعیت در لحظه T0

پردازش	Allocation (A,B,C)	Max (A,B,C)
T0	(0,1,0)	(7,5,3)
T1	(2,0,0)	(3,2,2)
T2	(3,0,2)	(9,0,2)
T3	(2,1,1)	(2,2,2)
T4	(0,0,2)	(4,3,3)

- **منابع آزاد اولیه (Available):** (2, 3, 3)

◆ محاسبه ماتریس Need

ماتریس Need با استفاده از فرمول زیر محاسبه می‌شود:

$$\text{Need}[i] = \text{Max}[i] - \text{Allocation}[i]$$

محاسبات	Need (A,B,C)	پردازه
(7-0, 5-1, 3-0)	(7,4,3)	T0
(3-2, 2-0, 2-0)	(1,2,2)	T1
(9-3, 0-0, 2-2)	(6,0,0)	T2
(2-2, 2-1, 2-1)	(0,1,1)	T3
(4-0, 3-0, 3-2)	(4,3,1)	T4

❖ اجرای Safety Algorithm

هدف: بررسی وجود یک ترتیب امن برای اجرای پروزه‌ها به گونه‌ای که همه بتوانند کامل شوند.

شروع: $Work = Available = (3, 3, 2)$

$Finish = [false, false, false, false, false]$

1. **T1**: $Need(1,2,2) \leq Work(3,3,2) \rightarrow$ **قابل اجرا** ✓

• پس از اجرا: $Work = Work + Allocation = (3,3,2) + (2,0,0) = (5,3,2)$

• $Finish[1] = true$

2. **T3**: $Need(0,1,1) \leq Work(5,3,2) \rightarrow$ **قابل اجرا** ✓

• پس از اجرا: $Work = (5,3,2) + (2,1,1) = (7,4,3)$

• $Finish[3] = true$

3. **T4**: $Need(4,3,1) \leq Work(7,4,3) \rightarrow$ **قابل اجرا** ✓

• پس از اجرا: $Work = (7,4,3) + (0,0,2) = (7,4,5)$

• $Finish[4] = true$

4. **T0**: $Need(7,4,3) \leq Work(7,4,5) \rightarrow$ **قابل اجرا** ✓

• پس از اجرا: $Work = (7,4,5) + (0,1,0) = (7,5,5)$

• $Finish[0] = true$

5. **T2**: $Need(6,0,0) \leq Work(7,5,5) \rightarrow$ **قابل اجرا** ✓

• پس از اجرا: $Work = (7,5,5) + (3,0,2) = (10,5,7)$

• $Finish[2] = true$

نتیجه 🎓

- سیستم در یک حالت امن قرار دارد.
- یک ترتیب امن (Safe Sequence) برای اجرای پروزه‌ها یافت شد: $\{T1, T3, T4, T0, T2\}$

🔑 نکات کلیدی

- تصمیم می‌گیرد کدام پروزه قابل اجراست (Available) $Need \leq Work$ با مقایسه‌ی Banker's Algorithm.
- اگر حداقل یک ترتیب امن برای اجرای همه پروزه‌ها وجود داشته باشد، سیستم در **Safe State** قرار دارد.
- در این مثال، همه‌ی پروزه‌ها می‌توانند بدون بن‌بست تکمیل شوند.
- مهم‌ترین محاسبات:

• $Need = Max - Allocation$

• بروزرسانی $Work$ بعد از اتمام هر پروزه: $Work = Work + Allocation$

Example: P₁ Request (1,0,2)

این اسلاید نشان می‌دهد چگونه الگوریتم Banker یک درخواست جدید منابع از طرف پردازش P₁ را بررسی می‌کند.

مرحله ۱: بررسی اولیه

درخواست کرده P₁:

$$\text{Request}_1 = (1, 0, 2)$$

شرط اول الگوریتم:

$$\text{Request}_1 \leq \text{Need}_1$$

از جدول: $\text{Need}_1 = (1, 2, 2)$

$$(1, 2, 2) \geq (1, 0, 2) \quad \checkmark$$

شرط دوم:

$$\text{Request}_1 \leq \text{Available}$$

$$(1, 0, 2) \leq (3, 3, 2) \Rightarrow \text{True} \quad \checkmark$$

مرحله ۲: تخصیص آزمایشی

اگر درخواست موقتاً به P₁ داده شود:

- $\text{Available} (2, 3, 0) = (1, 0, 2) - (3, 3, 2) = \text{جدید}$
- $\text{Allocation}[P_1] (3, 0, 2) = (1, 0, 2) + (2, 0, 0) = \text{جدید}$
- $\text{Need}[P_1] (0, 2, 0) = (1, 0, 2) - (1, 2, 2) = \text{جدید}$

مرحله ۳: اجرای Safety Algorithm

حالا بررسی می‌کنیم که آیا سیستم در حالت امن باقی می‌ماند یا نه. با شرایط جدید، یک توالی امن پیدا می‌شود:

$$\{T_1, T_3, T_4, T_0, T_2\}$$

این یعنی تخصیص درخواست P₁ سیستم را در حالت امن نگه می‌دارد.

سوالات ادامه

آیا درخواست (3,3,0) از طرف T₄ پذیرفته می‌شود؟

- بررسی اول: $\text{Request}_4 = (3, 3, 0) \leq \text{Need}_4 = (4, 3, 1)$ $\checkmark$ درست است.
- بررسی دوم: $\text{Available} = (2, 3, 0) \geq (3, 3, 0)$ $\times$ نادرست (نیاز به منبع A بیشتر از موجودی است).
- پس این درخواست رد می‌شود و T₄ باید منتظر بماند.

آیا درخواست (0,2,0) از طرف T₀ پذیرفته می‌شود؟

- بررسی اول: $\text{Need}_0 = (7, 4, 3) \geq (0, 2, 0)$ $\checkmark$ درست
- بررسی دوم: $\text{Available} = (2, 3, 0) \geq (0, 2, 0)$ $\checkmark$ درست
- سیستم بررسی حالت امن را انجام می‌دهد. اگر پس از تخصیص آزمایشی توالی امن وجود داشته باشد $\rightarrow$ درخواست پذیرفته می‌شود. در این مثال، این درخواست پذیرفته می‌شود.

- الگوریتم Banker هر درخواست را در دو مرحله بررسی می‌کند:
 1. آیا درخواست بیشتر از Need نیست؟
 2. آیا منابع کافی در Available وجود دارد؟
- سپس با **تخصیص آزمایشی** و اجرای **Safety Algorithm** مطمئن می‌شود سیستم وارد Unsafe State نشود.
- درخواست $P_1(1, 0, 2)$ پذیرفته شد چون سیستم امن باقی ماند.
- درخواست $T_4(3, 3, 0)$ رد شد چون بیشتر از منابع موجود بود.
- درخواست $T_0(0, 2, 0)$ پذیرفته شد چون بعد از تخصیص، سیستم هنوز Safe State داشت.

Example of Safe State

این اسلاید یک مثال کامل از **اجرای الگوریتم Banker** برای بررسی اینکه سیستم در حالت امن (Safe State) قرار دارد یا نه، نشان می‌دهد.

◆ داده‌های اولیه

- بردار کل منابع (Resource vector R): $R = (9, 3, 6)$
- بردار منابع موجود (Available vector V): $V = (0, 1, 1)$

ماتریس Claim (حداکثر نیاز هر پردازش)

Process	R1	R2	R3
P ₁	3	2	2
P ₂	6	1	3
P ₃	3	1	4
P ₄	4	2	2

ماتریس Allocation (مقدار تخصیص یافته فعلی)

Process	R1	R2	R3
P ₁	1	0	0
P ₂	6	1	2
P ₃	2	1	1
P ₄	0	0	2

ماتریس Need (محاسبه شده به صورت Claim – Allocation)

Process	R1	R2	R3
P ₁	2	2	2
P ₂	0	0	1
P ₃	1	0	3
P ₄	4	2	0

◆ اجرای Safety Algorithm

1. شروع با $Available = (0, 1, 1)$

- ✓ → P_2 : Need = (0, 0, 1) ≤ (0, 1, 1) بررسی
→ اجرا می‌شود. بعد از اتمام، منابع آزاد می‌شود:
 $V = (0, 1, 1) + (6, 1, 2) = (6, 2, 3)$

2. اکنون Available = (6, 2, 3)

- ✓ → P_1 : Need = (2, 2, 2) ≤ (6, 2, 3) بررسی
→ اجرا می‌شود. آزادسازی منابع:
 $V = (6, 2, 3) + (1, 0, 0) = (7, 2, 3)$

3. Available = (7, 2, 3)

- ✓ → P_3 : Need = (1, 0, 3) ≤ (7, 2, 3) بررسی
→ اجرا می‌شود. آزادسازی منابع:
 $V = (7, 2, 3) + (2, 1, 1) = (9, 3, 4)$

4. Available = (9, 3, 4)

- ✓ → P_4 : Need = (4, 2, 0) ≤ (9, 3, 4) بررسی
→ اجرا می‌شود. آزادسازی منابع:
 $V = (9, 3, 4) + (0, 0, 2) = (9, 3, 6)$

نتیجه 🎓

همه پردازها توانستند اجرا شوند. پس سیستم در حالت **Safe State** قرار دارد.
یک توالی امن برای اجرای پردازها: $\langle P_2, P_1, P_3, P_4 \rangle$

نکات کلیدی 🔑

- سیستم ابتدا منابع موجود را با نیاز پردازها مقایسه می‌کند.
- هر پردازهای که نیازش $\geq$ منابع موجود باشد می‌تواند اجرا شود.
- بعد از اجرا، منابع تخصیص یافته آزاد می‌شوند و به Available اضافه می‌شوند.
- اگر در نهایت همه پردازها بتوانند اجرا شوند → سیستم در **حالت امن** است.
- توالی امن در این مثال: $\langle P_2, P_1, P_3, P_4 \rangle$.

⚠ Example of Unsafe State

این اسلاید مثالی از حالتی است که اگر یک درخواست خاص پذیرفته شود، سیستم وارد حالت **Unsafe** می‌شود.

💠 داده‌های اولیه

- Resource Vector (کل منابع سیستم):** $R = (9, 3, 6)$
- Available Vector (منابع آزاد در حال حاضر):** $V = (1, 1, 2)$

Claim Matrix (حداکثر نیاز پردازها)

Process	R1	R2	R3
P_1	3	2	2
P_2	6	1	3
P_3	3	1	4
P_4	4	2	2

Allocation Matrix (مقدار تخصیص یافته فعلی)

Process	R1	R2	R3
P ₁	1	0	0
P ₂	5	1	1
P ₃	2	1	1
P ₄	0	0	2

Need Matrix (C - A) محاسبه شده به صورت

Process	R1	R2	R3
P ₁	2	2	2
P ₂	1	0	2
P ₃	1	0	3
P ₄	4	2	0

♦ درخواست جدید

پردازه P₁ درخواست می‌دهد: Request = (1, 0, 1)

بررسی مرحله اول:

آیا Request ≤ Available ؟

(1, 1, 2) ≥ (1, 0, 1) 

پس درخواست از نظر منابع آزاد قابل پذیرش است.

♦ به‌روزرسانی ماتریس‌ها بعد از پذیرش درخواست

- Allocation جدید برای P₁: (1, 0, 0) + (1, 0, 1) = (2, 0, 1)
- Need جدید برای P₁: (2, 2, 2) - (1, 0, 1) = (1, 2, 1)
- جدید Available (0, 1, 1) = (1, 0, 1) - (1, 1, 2)

♦ بررسی Safe State

با Available جدید (0, 1, 1) :

- P₁: Need = (1, 2, 1) → ❌ (کافی نیست)
- P₂: Need = (1, 0, 2) → ❌ (بیشتر است R3 نیاز)
- P₃: Need = (1, 0, 3) → ❌ (بیشتر است R1 و R3 نیاز)
- P₄: Need = (4, 2, 0) → ❌ (بیشتر است R1 نیاز)

هیچ پردازهای نمی‌تواند ادامه پیدا کند → سیستم Deadlock محتمل → حالت Unsafe.

🔑 نتیجه

- پذیرش درخواست (1, 0, 1) توسط P₁، سیستم را از حالت امن خارج کرده و وارد حالت Unsafe می‌کند.
- در Unsafe State هنوز لزوماً Deadlock رخ نداده است، اما امکان وقوع آن وجود دارد چون توالی امنی وجود ندارد.

◆ Deadlock Detection

در مقابل **Deadlock Prevention** و **Deadlock Avoidance**، در روش **Deadlock Detection** سیستم اجازه می‌دهد وارد بن‌بست شود و سپس با یک الگوریتم، آن را **تشخیص داده** و با یک **Recovery Scheme** از آن خارج شود.

1. ✓ Detection Algorithm

الف) برای سیستم تک‌منبعی (Single Instance of Each Resource Type)

- از **Graph Reduction Algorithm** استفاده می‌شود.
- منابع و پردازنده‌ها به صورت گراف اختصاص داده می‌شوند.
- اگر در گراف یک **دور (Cycle)** وجود داشته باشد → **Deadlock رخ داده است**.

ب) برای سیستم چندمنبعی (Multiple Instances of Resources)

- مشابه بانکدار (Banker's Algorithm) ولی با هدف **تشخیص** نه جلوگیری.
- ماتریس‌ها:
- Available**: منابع آزاد
- Allocation**: تخصیص یافته
- Request**: درخواست‌های باقی‌مانده
- الگوریتم بررسی می‌کند که آیا می‌توان پردازنده‌ها را یکی یکی تکمیل و آزاد کرد یا خیر.
- اگر پردازنده‌هایی باقی بمانند که نیازشان قابل برآورده کردن نیست → این‌ها در **Deadlock** هستند.

2. ✓ Recovery Scheme

وقتی **Deadlock** شناسایی شد، باید یکی از روش‌های زیر اعمال شود:

◆ الف) Process Termination

1. Abort all Deadlocked Processes

- همه پردازنده‌های درگیر را متوقف می‌کنیم.
- ساده ولی پرهزینه.

2. Abort One Process at a Time Until Deadlock is Broken

- انتخاب قربانی (Victim Selection) بر اساس:
- اولویت پردازنده
- مدت زمان اجرای پردازنده
- تعداد منابع مورد استفاده
- مدت زمان لازم تا تکمیل

◆ ب) Resource Preemption

- گرفتن منابع از بعضی پردازنده‌ها و اختصاص به دیگران.
- نیازمند:

- Victim Selection**: انتخاب پردازنده‌ای که منابعش گرفته می‌شوند.
- Rollback**: برگرداندن پردازنده به حالت قبلی.
- Starvation Issue**: اگر همیشه یک پردازنده قربانی شود → نیاز به یک سیاست **عادلانه**.

🔑 خلاصه

- Detection Algorithm**: با گراف یا ماتریس منابع **Deadlock** پیدا کردن.
- Recovery Scheme**: با توقف پردازنده‌ها یا گرفتن منابع **Deadlock** رفع.

- مزیت: بهره‌وری بالاتر از جلوگیری کامل.
- عیب: سیستم می‌تواند برای مدتی در Deadlock گیر کند.

◆ Deadlock Detection with Single Instance of Each Resource Type

در این حالت، چون از هر نوع منبع فقط یک نسخه (Single Instance) وجود دارد، الگوریتم ساده‌تر می‌شود.

روش کار: Wait-For Graph (WFG) ✓

1. ساخت گراف:
 - گره‌ها (Nodes): پردازنده‌ها (Threads/Processes).
 - یال‌ها (Edges):
 - اگر پردازنده T_i در انتظار منبعی باشد که هم‌اکنون توسط پردازنده T_j نگه داشته شده، یک یال از $T_i \rightarrow T_j$ رسم می‌شود.
 - به این گراف می‌گوییم **Wait-For Graph**.
2. تشخیص بن‌بست:
 - الگوریتم به صورت دوره‌ای (Periodically) روی گراف اجرا می‌شود.
 - اگر در گراف یک چرخه (Cycle) وجود داشته باشد $\rightarrow$ **Deadlock رخ داده است**.
 - اگر هیچ چرخه‌ای نباشد $\rightarrow$ سیستم در وضعیت امن است.
3. پیچیدگی زمانی (Time Complexity):
 - الگوریتم پیدا کردن چرخه در گراف نیاز به $O(n^2)$ عملیات دارد، که در آن n تعداد گره‌ها (پردازنده‌ها) است.

مثال ساده 🔑

- فرض کنید سه پردازنده داریم: P_1, P_2, P_3
 - وابستگی‌ها:
 - $P_1 \rightarrow P_2$ است $\rightarrow$ یال $P_1 \rightarrow P_2$ در انتظار منبعی است که در اختیار P_1
 - $P_2 \rightarrow P_3$ است $\rightarrow$ یال $P_2 \rightarrow P_3$ در انتظار منبعی است که در اختیار P_2
 - $P_3 \rightarrow P_1$ است $\rightarrow$ یال $P_3 \rightarrow P_1$ در انتظار منبعی است که در اختیار P_3
- 👉 در اینجا یک چرخه $(P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_1)$ داریم $\rightarrow$ پس **Deadlock رخ داده است**.

خلاصه 📌

- **Single Instance** $\rightarrow$ استفاده از **Wait-For Graph**.
- تشخیص **Deadlock** $\rightarrow$ وجود چرخه در گراف.
- هزینه محاسباتی $\rightarrow O(n^2)$.

◆ Deadlock Detection

در مقابل **Deadlock Prevention** و **Deadlock Avoidance**، در روش **Deadlock Detection** سیستم اجازه می‌دهد وارد بن‌بست شود و سپس با یک الگوریتم، آن را تشخیص داده و با یک **Recovery Scheme** از آن خارج شود.

1. ✓ Detection Algorithm

(الف) برای سیستم تک‌منبعی (Single Instance of Each Resource Type)

- از **Graph Reduction Algorithm** استفاده می‌شود.
- منابع و پردازنده‌ها به صورت گراف اختصاص داده می‌شوند.
- اگر در گراف یک **دور (Cycle)** وجود داشته باشد → **Deadlock رخ داده است**.

ب) برای سیستم چندمنبعی (Multiple Instances of Resources)

- مشابه بانکدار (Banker's Algorithm) ولی با هدف **تشخیص** نه جلوگیری.
- ماتریس‌ها:
- **Available**: منابع آزاد
- **Allocation**: تخصیص یافته
- **Request**: درخواست‌های باقی‌مانده
- الگوریتم بررسی می‌کند که آیا می‌توان پردازنده‌ها را یکی یکی تکمیل و آزاد کرد یا خیر.
- اگر پردازنده‌هایی باقی بمانند که نیازشان قابل برآورده کردن نیست → این‌ها در **Deadlock** هستند.

2. Recovery Scheme

وقتی Deadlock شناسایی شد، باید یکی از روش‌های زیر اعمال شود:

♦ الف) Process Termination

1. Abort all Deadlocked Processes

- همه پردازنده‌های درگیر را متوقف می‌کنیم.
- ساده ولی پرهزینه.

2. Abort One Process at a Time Until Deadlock is Broken

- انتخاب قربانی (Victim Selection) بر اساس:
- اولویت پردازنده
- مدت زمان اجرای پردازنده
- تعداد منابع مورد استفاده
- مدت زمان لازم تا تکمیل

♦ ب) Resource Preemption

- گرفتن منابع از بعضی پردازنده‌ها و اختصاص به دیگران.
- نیازمند:

1. **Victim Selection**: انتخاب پردازنده‌ای که منابعش گرفته می‌شوند.
2. **Rollback**: برگرداندن پردازنده به حالت قبلی.
3. **Starvation Issue**: اگر همیشه یک پردازنده قربانی شود → نیاز به یک سیاست **عادلانانه**.

خلاصه

- با گراف یا ماتریس منابع Deadlock پیدا کردن.
- **Detection Algorithm**: با گراف یا ماتریس منابع Deadlock پیدا کردن.
- **Recovery Scheme**: با توقف پردازنده‌ها یا گرفتن منابع Deadlock رفع.
- مزیت: بهره‌وری بالاتر از جلوگیری کامل.
- عیب: سیستم می‌تواند برای مدتی در Deadlock گیر کند.

♦ Deadlock Detection – Single Instance of Each Resource Type

در این حالت فرض می‌کنیم که **از هر نوع منبع فقط یک نمونه وجود دارد**. بنابراین روش تشخیص Deadlock ساده‌تر از حالتی است که چند نمونه (Multiple Instances) داریم.

1. Wait-For Graph

تعریف:

- در این مدل از یک **Wait-For Graph (WFG)** استفاده می‌شود.
- **گره‌ها (Nodes)** نشان‌دهنده‌ی **پردازه‌ها (Threads/Processes)** هستند.
- یک **پال جهت‌دار (Directed Edge)** از $T_i \rightarrow T_j$ به این معناست که پردازه T_i منتظر پردازه T_j است تا منبعی را آزاد کند.

به عبارت دیگر:

- اگر T_i منبعی را لازم داشته باشد که در حال حاضر در اختیار T_j است، یک پال از T_i به T_j رسم می‌کنیم.

2. تشخیص Deadlock با گراف

- سیستم به صورت **دوره‌ای (Periodically)** الگوریتم تشخیص را اجرا می‌کند.
- الگوریتم کافیت بررسی کند که آیا در گراف یک **چرخه (Cycle)** وجود دارد یا خیر.

◆ نتیجه:

- اگر چرخه‌ای در گراف نباشد $\rightarrow$ سیستم **بدون Deadlock** است.
- اگر چرخه‌ای وجود داشته باشد $\rightarrow$ پردازه‌های داخل آن چرخه در **بن‌بست (Deadlock)** هستند.

3. پیچیدگی الگوریتم

- برای پیدا کردن چرخه در یک گراف با n گره:
- زمان اجرای الگوریتم تقریباً $O(n^2)$ است.
- این کار با استفاده از **DFS (Depth-First Search)** یا **الگوریتم‌های cycle detection** انجام می‌شود.

4. مثال آموزشی

فرض کنید 3 پردازه داریم: P_1, P_2, P_3

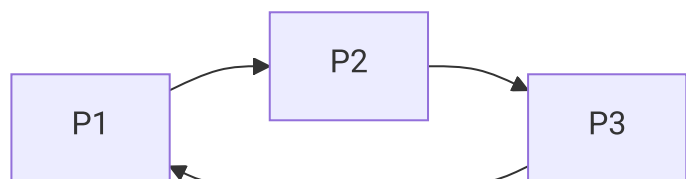
- $P_1 \rightarrow P_2$ است $\rightarrow$ P_1 منتظر منبعی است که در اختیار P_2 است
- $P_2 \rightarrow P_3$ است $\rightarrow$ P_2 منتظر منبعی است که در اختیار P_3 است
- $P_3 \rightarrow P_1$ است $\rightarrow$ P_3 منتظر منبعی است که در اختیار P_1 است

🔥 در این حالت گراف یک چرخه دارد:

$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_1$

این یعنی پردازه‌ها در **بن‌بست** قرار دارند.

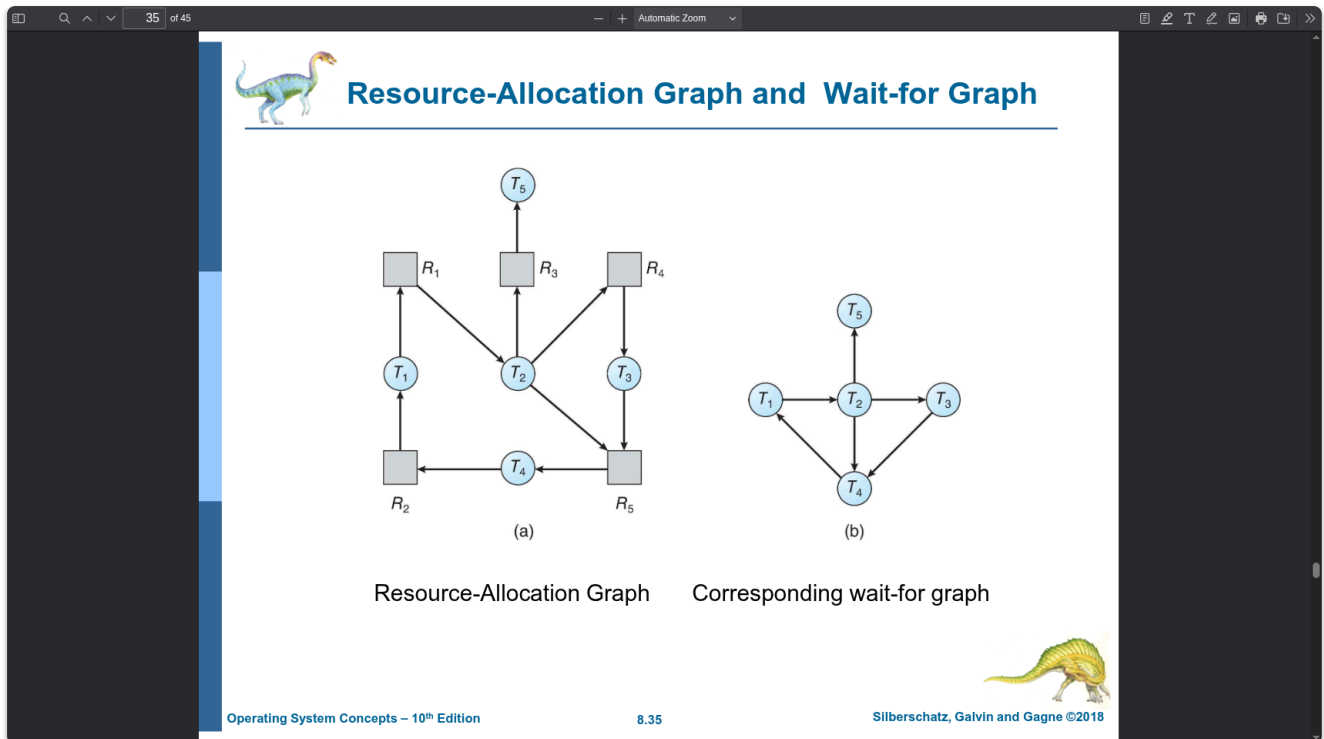
5. نمایش گراف با Mermaid



این دیاگرام نشان می‌دهد که بین پردازه‌های P_1, P_2, P_3 یک چرخه وجود دارد و بنابراین Deadlock رخ داده است.

نکات کلیدی

- در حالت یک نمونه از هر منبع، تشخیص Deadlock با Wait-For Graph انجام می‌شود.
- وجود یک چرخه در گراف معادل وقوع Deadlock است.
- الگوریتم تشخیص چرخه در گراف دارای پیچیدگی $O(n^2)$ است.
- این روش ساده، اما در سیستم‌های بزرگ ممکن است هزینه‌بر باشد.



Resource-Allocation Graph و Wait-for Graph

این اسلاید نشان می‌دهد که چگونه می‌توان یک Resource-Allocation Graph (RAG) را به Wait-for Graph (WFG) تبدیل کرد تا تشخیص Deadlock ساده‌تر شود.

1. Resource-Allocation Graph (RAG)

- گره‌های دایره‌ای $\rightarrow$ پردازنده‌ها (T_1, T_2, T_3, T_4, T_5)
- گره‌های مربعی $\rightarrow$ منابع (R_1, R_2, R_3, R_4, R_5)
- یال جهت‌دار از پردازنده به منبع $\rightarrow$ پردازنده منبعی را درخواست کرده است.
- یال جهت‌دار از منبع به پردازنده $\rightarrow$ منبع به پردازنده‌ای اختصاص داده شده است.

در تصویر (a):

- مثلاً $T_1 \rightarrow R_1$ یعنی پردازنده T_1 منتظر است تا منبع R_1 آزاد شود.
- است $T_1 \rightarrow R_2$ هم‌اکنون در اختیار R_2 یعنی منبع T_1 است.

2. Wait-for Graph (WFG)

برای ساده‌سازی، منابع حذف می‌شوند و فقط پردازنده‌ها باقی می‌مانند:

- اگر پردازنده T_i در انتظار منبعی باشد که در اختیار پردازنده T_j است، یک یال مستقیم $T_i \rightarrow T_j$ در WFG رسم می‌شود.

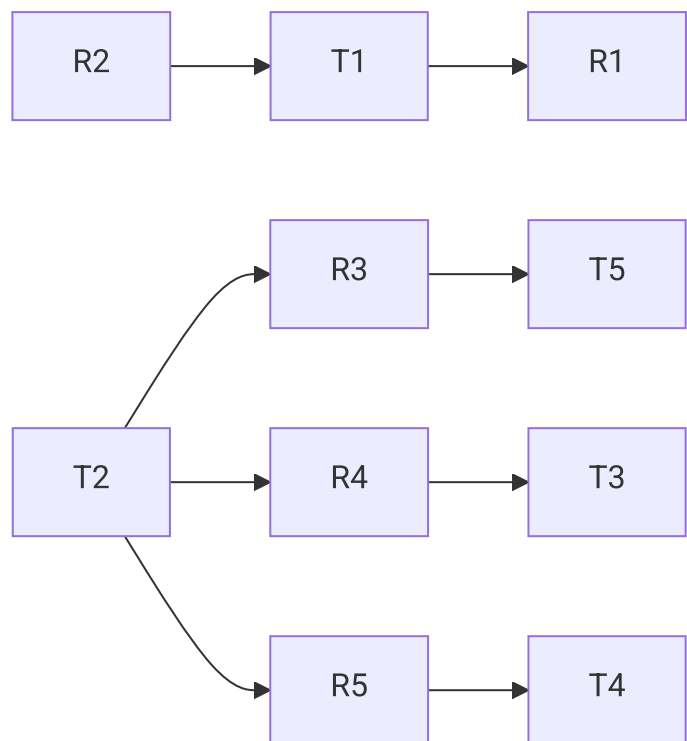
- $T_1 \rightarrow T_2 T_1 \rightarrow T_2$: یعنی $T_1 T_1$ در اختیار است که در $T_2 T_2$ منتظر منبعی است.
- $T_2 \rightarrow T_4 T_2 \rightarrow T_4$: یعنی $T_2 T_2$ در اختیار است که در $T_4 T_4$ منتظر منبعی است.
- $T_3 \rightarrow T_4 T_3 \rightarrow T_4$: یعنی $T_3 T_3$ در اختیار است که در $T_4 T_4$ منتظر منبعی است.
- $T_2 \rightarrow T_5 T_2 \rightarrow T_5$: یعنی $T_2 T_2$ در اختیار است که در $T_5 T_5$ منتظر منبعی است.

3. اهمیت تبدیل WFG به RAG ✓

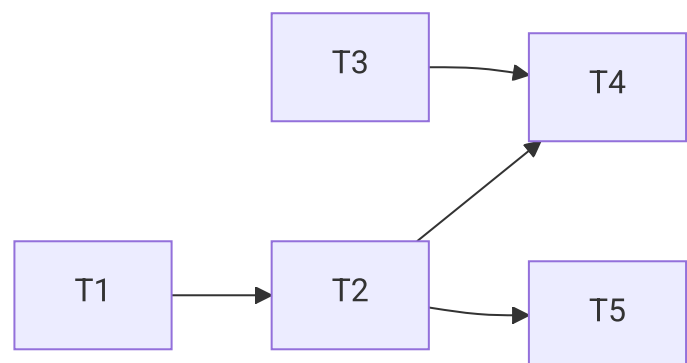
- در سیستم‌هایی که **هر منبع فقط یک نمونه دارد**، کافی است به جای نگهداری RAG کامل، از WFG استفاده کنیم.
- **مزیت اصلی**: تشخیص Deadlock به یافتن **چرخه (Cycle)** در WFG خلاصه می‌شود.

4. مثال با دیاگرام Mermaid ✓

Resource-Allocation Graph:



Wait-for Graph (معادل ساده‌شده):



- شامل هر دو نوع گره (پردازه + منبع) است **RAG**.
- برای یک نمونه از هر منبع، می‌توان آن را به **WFG** ساده کرد.
- در **WFG**:
- یال از TiT_i به $TiT_i \Leftarrow TjT_j$ منتظر منبعی است که در اختیار TjT_j است.
- وجود یک چرخه در **WFG** $\Leftrightarrow$ وجود **Deadlock**.

◆ Deadlock Detection – Several Instances of a Resource Type

وقتی هر نوع منبع چندین نمونه (instances) دارد، تشخیص Deadlock نسبت به حالت تک‌نمونه‌ای پیچیده‌تر می‌شود. در این حالت دیگر **Wait-for Graph** ساده کافی نیست و باید از یک روش مبتنی بر **ماتریس‌ها** (مشابه الگوریتم Banker) استفاده کنیم.

1. ✓ ساختارهای داده موردنیاز

برای تشخیص Deadlock، سه ساختار اصلی نگهداری می‌شوند:

◆ a) Available (بردار منابع آزاد)

- یک بردار به طول m (تعداد انواع منابع).
- مقدار خانمی $Available[j]$ = تعداد منابع آزاد از نوع RjR_j .

◆ b) Allocation (ماتریس تخصیص)

- یک ماتریس $n \times m$ (تعداد پردازها، n = تعداد انواع منابع).
- مقدار $Allocation[i][j]$ = تعداد منابع نوع RjR_j که در حال حاضر به پردازه TiT_i تخصیص داده شده است.

◆ c) Request (ماتریس درخواست)

- یک ماتریس $n \times m$.
- مقدار $Request[i][j] = k$ یعنی پردازه TiT_i در حال حاضر k نمونه از منبع RjR_j را درخواست کرده است.

2. ✓ الگوریتم تشخیص Deadlock

الگوریتم شبیه الگوریتم Banker است اما به جای بررسی ایمنی، بررسی می‌کنیم که آیا پردازه‌هایی وجود دارند که نیازشان برآورده نمی‌شود.

مراحل:

1. Initialize:

- $Work = Available$ (کپی از بردار منابع آزاد)
- $Finish[i] = false$ برای همه‌ی پردازها

2. Find:

- پردازهای پیدا کن که $Finish[i] = false$ باشد و $Request_i \leq Work$.
- اگر چنین پردازهای وجود داشت، فرض کن اجرا می‌شود.

3. Execute:

- منابعش را آزاد می‌کند: $Work = Work + Allocation_i$
- مقدار $Finish[i] = true$

4. Repeat:

- این مراحل تکرار می‌شود تا دیگر پردازهای یافت نشود.

5. Check Deadlock:

- اگر بعد از پایان الگوریتم هنوز پردازه‌هایی باشند که $Finish[i] = false$ ، آن‌ها در Deadlock گیر کرده‌اند.

3. مثال عددی (آموزشی) ✓

فرض کنید 3 نوع منبع داریم: A, B, C

- Available = (2, 1, 0)

ماتریس Allocation:

Process	A	B	C
P1	1	0	1
P2	1	1	0
P3	1	0	1

ماتریس Request:

Process	A	B	C
P1	0	1	0
P2	1	0	1
P3	0	1	0

♦ اجرای الگوریتم:

- Work = (2,1,0)
- P1: Request = (0,1,0) ≤ Work → اجرا می‌شود → Work = (3,1,1)
- P2: Request = (1,0,1) ≤ Work → اجرا می‌شود → Work = (4,2,1)
- P3: Request = (0,1,0) ≤ Work → اجرا می‌شود → Work = (5,2,2)

👉 همه پردازش‌ها **Deadlock** → Finish = true وجود ندارد.

🔑 نکات کلیدی

- در حالت چند نمونه‌ای از هر منبع باید از سه ساختار داده‌ی Available, Allocation, Request استفاده شود.
- الگوریتم مشابه بانکدار است اما هدف آن صرفاً تشخیص پردازش‌هایی است که گیر کرده‌اند.
- خروجی الگوریتم: پردازش‌هایی با Finish = false همان پردازش‌های Deadlock هستند.

♦ Deadlock Detection Algorithm (Several Instances of Each Resource Type)

الگوریتم تشخیص بن‌بست در حالتی که هر نوع منبع چندین نمونه دارد، از بردارها و ماتریس‌هایی مثل Available, Allocation, Request استفاده می‌کند. هدف این الگوریتم شناسایی پردازش‌هایی است که در وضعیت بن‌بست قرار گرفته‌اند.

✓ مراحل الگوریتم

1. مقداردهی اولیه (Initialization)

- تعریف دو بردار:
- Work → (تعداد انواع منابع) mm از نوع طول
- Finish → (تعداد پردازش‌ها) nn از نوع طول
- مقداردهی:

- `Work = Available` (بردار منابع آزاد)

• برای هر پردازش T_i :

• اگر $Allocation_i \neq 0$ → یعنی پردازش در حال استفاده از منابع است → `Finish[i] = false`

• اگر $Allocation_i = 0$ → یعنی هیچ منبعی به آن تخصیص نیافته → `Finish[i] = true`

2. جستجوی پردازش مناسب (Search Step)

• یک اندیس i پیدا کن که:

1. `Finish[i] == false`

2. `Request[i] ≤ Work` (یعنی نیاز پردازش قابل برآورده شدن با منابع فعلی است)

• اگر چنین پردازشی پیدا نشد، برو به مرحله 4.

3. شبیه‌سازی اجرای پردازش (Execution Step)

• فرض می‌کنیم پردازش T_i اجرا شده و منابعش آزاد می‌شوند: `Work = Work + Allocation_i`

• مقدار `Finish[i] = true` قرار داده می‌شود.

• سپس الگوریتم دوباره به مرحله 2 بازمی‌گردد.

4. تشخیص بن‌بست (Deadlock Detection)

• اگر در پایان الگوریتم هنوز برای بعضی i : `Finish[i] == false` برقرار باشد، یعنی پردازش T_i در وضعیت بن‌بست است.

• مجموعه پردازش‌هایی که مقدار `Finish` آن‌ها برابر `false` باقی مانده است، دقیقاً پردازش‌های بن‌بست‌شده هستند.

✓ تحلیل پیچیدگی الگوریتم

• الگوریتم نیاز دارد تا در بدترین حالت تمام پردازش‌ها و منابع را بررسی کند.

• زمان اجرای آن: $O(m \times n^2)$ که در آن:

- m : تعداد انواع منابع
- n : تعداد پردازش‌ها

✓ شبه‌کد (Pseudocode)

```
Detection-Algorithm(Available, Allocation, Request)
```

```
1. Work = Available
```

```
2. For i = 1 to n do
```

```
    if Allocation[i] ≠ 0 then
```

```
        Finish[i] = false
```

```
    else
```

```
        Finish[i] = true
```

```
3. While exists an i such that (Finish[i] == false) and (Request[i] ≤ Work) do
```

```
    Work = Work + Allocation[i]
```

```
    Finish[i] = true
```

```
4. If exists an i such that Finish[i] == false then
```

```
    System is in Deadlock
```

```
    All  $T_i$  with Finish[i] == false are deadlocked
```

```
Else
```

```
    No Deadlock
```

✓ نکات مهم

- این الگوریتم اجازه می‌دهد سیستم وارد **بن‌بست** شود، سپس آن را تشخیص دهد.
- بعد از تشخیص، سیستم باید یک **Recovery Scheme** (طرح بازیابی) داشته باشد، مثل:
 - خاتمه دادن به برخی پردازها
 - گرفتن منابع از پردازها

الگوریتم تشخیص بن‌بست (Deadlock Detection Algorithm)

توضیح ساده و دقیق

این اسلاید مربوط به الگوریتم **تشخیص بن‌بست** (Deadlock Detection) است که در سیستم‌عامل‌ها برای بررسی وجود وضعیت بن‌بست بین فرآیندها استفاده می‌شود. این الگوریتم زمانی کاربرد دارد که **مکانیزم جلوگیری یا اجتناب از بن‌بست** پیاده‌سازی نشده باشد و سیستم بخواهد تشخیص دهد آیا بن‌بست رخ داده است یا خیر.

مراحل الگوریتم:

1. تعریف بردارها:

- Work (منابع آزاد موجود) مقدار اولیه برابر با m (تعداد منابع) برداری به طول m : $Work$
- نشان می‌دهد آیا یک فرآیند می‌تواند کارش را تمام کند یا خیر، (تعداد فرآیندها) n آرایه‌ای به طول n : $Finish$

2. مقداردهی اولیه:

- برای هر فرآیند i :
 - اگر $Allocation[i] \neq 0$ (یعنی فرآیند در حال استفاده از منابع است) $Finish[i] = false$
 - اگر هیچ منبعی نگرفته $(Allocation[i] = 0)$ $Finish[i] = true$

3. یافتن فرآیند قابل اجرا:

- بررسی کن آیا فرآیندی i وجود دارد که:
 - $Request[i] \leq Work$ و
 - اگر چنین فرآیندی پیدا نشد $\rightarrow$ برو به مرحله 4.

4. به‌روزرسانی وضعیت:

- اگر فرآیند پیدا شد:
 - منابع تخصیص داده شده به آن را آزاد کن: $Work = Work + Allocation[i]$
 - وضعیت آن را تمام شده علامت بزن: $Finish[i] = true$
 - سپس دوباره به مرحله 2 برگرد.

5. تشخیص بن‌بست:

- در انتها، اگر هنوز فرآیندی با $Finish[i] = false$ وجود داشته باشد $\rightarrow$ آن فرآیندها در بن‌بست هستند.
- در غیر این صورت $\rightarrow$ سیستم بدون بن‌بست است.

6. پیچیدگی زمانی:

- این الگوریتم برای تشخیص بن‌بست به تعداد عملیاتی در حدود $O(m \times n^2)$ نیاز دارد.

ارتباط با کتاب

- (تشخیص و ریکوری) **Deadlocks فصل – Silberschatz, Galvin, Gagne**
- این الگوریتم مشابه **Banker's algorithm** است اما به جای اجتناب، برای **تشخیص** به‌کار می‌رود.

مثال آموزشی کوتاه

فرض کن در یک سیستم:

- تعداد منابع $(m) = 1$ نوع** (مثلاً پرینتر).
- تعداد فرآیندها $(n) = 3$.**

- Available = 1
- Allocation = [1, 0, 1]
- Request = [0, 1, 0]

اجرای الگوریتم:

1. Work = Available = 1
Finish = [false, true, false]

2. پیدا کردن فرآیندی با $Request \leq Work$:

- P1: Request=0 $\leq$ 1 → انتخاب می‌شود.
Work = Work + Allocation[1] = 1+1=2, Finish[1]=true
- P2: Request=1 $\leq$ 2 → انتخاب می‌شود.
Work = 2+0=2, Finish[2]=true
- P3: Request=0 $\leq$ 2 → انتخاب می‌شود.
Work=2+1=3, Finish[3]=true

3. همه‌ی Finish=true → **بن‌بست وجود ندارد**.

نکات کلیدی

- الگوریتم تشخیص بن‌بست شبیه الگوریتم بانکدار است اما برای **تشخیص**، نه **اجتناب**.
- از دو بردار Work و Finish استفاده می‌کند.
- فرآیندهایی که در نهایت Finish=false باقی بمانند، در **بن‌بست** هستند.
- پیچیدگی زمانی: $O(m \times n^2)$.

مثال الگوریتم تشخیص بن‌بست (Deadlock Detection Example)

این اسلاید یک نمونه عملی از اجرای **الگوریتم تشخیص بن‌بست** را نشان می‌دهد. ما ماتریس‌های زیر داریم:

- **Request matrix (Q)**: نشان می‌دهد هر فرآیند چه مقدار منابع دیگر نیاز دارد تا کارش تمام شود.
- **Allocation matrix (A)**: نشان می‌دهد در حال حاضر چه منابعی به هر فرآیند تخصیص داده شده‌اند.
- **Resource vector**: کل منابع موجود در سیستم.
- **Available vector**: منابع آزاد باقی‌مانده در سیستم.

داده‌های مثال

Request Matrix Q

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	0	0	0
P3	0	0	0	0	1
P4	1	0	1	0	1

Allocation Matrix A

	R1	R2	R3	R4	R5
P1	1	0	1	0	0

	R1	R2	R3	R4	R5
P2	1	0	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

بردار منابع و موجودی

- **Resource Vector:** [2, 1, 1, 2, 1]
- **Available Vector:** [0, 0, 0, 0, 1]

اجرای الگوریتم

مرحله 1: مقداردهی اولیه

- $Work = Available = [0, 0, 0, 0, 1]$
- $Finish = [false, false, false, false]$ (ولی ما آن را هم در نظر می‌گیریم، چون همه‌ی $P4 \neq 0$ به جز Allocation)

مرحله 2: پیدا کردن فرآیند قابل اجرا

- **P1:** $Request[1] = [0, 1, 0, 0, 1] \rightarrow$ نمی‌تواند اجرا شود، اما $Work[2]=0$ نیاز به R2 دارد.
- **P2:** $Request[2] = [0, 0, 0, 0, 0] \leq Work \rightarrow$ قابل اجرا است.

- اجرا می‌شود، منابع آزاد می‌شوند:

$$Work = Work + Allocation[2] = [0, 0, 0, 0, 1] + [1, 0, 0, 0, 0] = [1, 0, 0, 0, 1]$$

$$Finish[2] = true$$

مرحله 3: دوباره جستجو

- **P1:** $Request[1] = [0, 1, 0, 0, 1] \rightarrow$ هنوز نیاز به R2 دارد که موجود نیست.
- **P3:** $Request[3] = [0, 0, 0, 0, 1] \leq Work[1, 0, 0, 0, 1] \rightarrow$ می‌تواند اجرا شود.

- اجرا می‌شود، منابع آزاد می‌شوند:

$$Work = [1, 0, 0, 0, 1] + [0, 0, 0, 1, 0] = [1, 0, 0, 1, 1]$$

$$Finish[3] = true$$

مرحله 4: دوباره جستجو

- **P1:** $Request[1] = [0, 1, 0, 0, 1] \rightarrow$ دارد R2 هنوز نیاز به R2 دارد.
- **P4:** $Request[4] = [1, 0, 1, 0, 1]$
 - $Work = [1, 0, 0, 1, 1] \rightarrow$ پس نمی‌تواند اجرا شود، $Work[3]=0$ (ولی) نیاز دارد R3.

مرحله 5: توقف

- تنها P1 و P4 باقی مانده‌اند که نمی‌توانند اجرا شوند.

نتیجه

- فرآیندهای P1 و P4 در وضعیت **بن‌بست (Deadlocked)** قرار دارند.
- فرآیندهای P2 و P3 می‌توانند اجرای خود را کامل کنند.

نکات کلیدی

- الگوریتم تشخیص بن‌بست با بردارهای Request, Allocation, Available کار می‌کند.
- فرآیندهایی که در نهایت $Finish = false$ باقی می‌مانند، در بن‌بست هستند.

- در این مثال، P1 و P4 بن بست دارند، در حالی که P2 و P3 آزاد شدند.

مثال الگوریتم تشخیص بن بست (Detection Algorithm Example)

این مثال نشان می‌دهد که چگونه الگوریتم تشخیص بن بست در شرایط مختلف عمل می‌کند. ما ۵ پردازش (T0 تا T4) و ۳ نوع منبع (A, B, C) داریم:

- تعداد کل منابع:

- A = 7
- B = 2
- C = 6

Snapshot در زمان T0

جدول تخصیص و درخواست

Request (A B C)	Allocation (A B C)	پردازش
0 0 0	0 1 0	T0
2 0 2	2 0 0	T1
0 0 0	3 0 3	T2
1 0 0	2 1 1	T3
0 0 2	0 0 2	T4

Available = (0, 0, 0)

اجرای الگوریتم (مرحله به مرحله)

مرحله 1

- Work = (0,0,0), Finish = [false,false,false,false,false]

- بررسی پردازش‌ها:

- T0: Request = (0,0,0) ≤ Work → می‌تواند اجرا شود.

- اجرای T0:

- Work = Work + Allocation(T0) = (0,0,0) + (0,1,0) = (0,1,0)
- Finish[T0] = true

مرحله 2

- Work = (0,1,0)

- بررسی پردازش‌ها:

- T2: Request = (0,0,0) ≤ Work → می‌تواند اجرا شود.

- اجرای T2:

- Work = (0,1,0) + (3,0,3) = (3,1,3)
- Finish[T2] = true

مرحله 3

- Work = (3,1,3)

- بررسی پردازش‌ها:

- T3: Request = (1,0,0) ≤ Work → می‌تواند اجرا شود.

• اجرای T3:

- $Work = (3,1,3) + (2,1,1) = (5,2,4)$
- $Finish[T3] = true$

مرحله 4

- $Work = (5,2,4)$

• بررسی پردازها:

- $T1: Request = (2,0,2) \leq Work \rightarrow$ می‌تواند اجرا شود.

• اجرای T1:

- $Work = (5,2,4) + (2,0,0) = (7,2,4)$
- $Finish[T1] = true$

مرحله 5

- $Work = (7,2,4)$

• بررسی پردازها:

- $T4: Request = (0,0,2) \leq Work \rightarrow$ می‌تواند اجرا شود.

• اجرای T4:

- $Work = (7,2,4) + (0,0,2) = (7,2,6)$
- $Finish[T4] = true$

نتیجه وضعیت اول

- دنباله اجرای ایمن: $\langle T0, T2, T3, T1, T4 \rangle$
- همه‌ی Finish‌ها $true \rightarrow$ هیچ بن‌بستی وجود ندارد.

تغییر شرایط (صفحه بعدی اسلاید)

فرض کنید T2 درخواست یک منبع اضافه از نوع C کند.

جدید Request

پردازه	Request (A B C)
T0	0 0 0
T1	2 0 2
T2	0 0 1
T3	1 0 0
T4	0 0 2

$Available = (0,0,0)$

اجرای الگوریتم در حالت جدید

- $T0$ می‌تواند اجرا شود ($Request=0$) $\rightarrow Work = (0,1,0)$

- اما پس از آزادسازی منابع T0، مقدار $Work = (0,1,0)$ خواهد بود.
- بررسی بقیه پردازها:

- نیازمند $(2,0,2)$ $\rightarrow$ کافی نیست T1.
- نیازمند $(0,0,1)$ $\rightarrow$ کافی نیست T2.

- نیازمند $(1,0,0)$ → کافی نیست T3.
- نیازمند $(0,0,2)$ → کافی نیست T4.

هیچ کدام از این پردازنده‌ها قابل اجرا نیستند → **بن‌بست وجود دارد**.

نتیجه وضعیت دوم

- پردازنده‌های درگیر بن‌بست: **T1, T2, T3, T4**
- فقط T0 توانست اجرا شود و منابعش آزاد شد.

نکات کلیدی

- در حالت اول، همه پردازنده‌ها می‌توانستند تکمیل شوند (بدون بن‌بست).
- در حالت دوم (وقتی T2 درخواست بیشتری کرد)، منابع کافی نبود و بن‌بست رخ داد.
- این نشان می‌دهد **یک تغییر کوچک در درخواست پردازنده‌ها** می‌تواند وضعیت سیستم را از **Safe State** به **Deadlock State** تغییر دهد.

Detection Algorithm Usage

نکته اصلی

الگوریتم تشخیص بن‌بست (Deadlock Detection Algorithm) فقط **وجود بن‌بست** رو مشخص می‌کنه؛ اما این‌که **چه زمانی** و **هر چند وقت** **یک بار** باید فراخوانی بشه، خودش یه تصمیم مهم طراحی سیستم هست.

۱. زمان و دفعات اجرای الگوریتم

- بستگی داره به اینکه:
 - **چه قدر احتمال وقوع بن‌بست زیاده؟**
 - اگه سیستم معمولاً منابع کافی داره و درخواست‌ها پراکنده‌ست → می‌شه کمتر اجراش کرد.
 - اگه سیستم شدیداً رقابتیه (مثلاً دیتابیس یا سیستم بلادرنگ) → باید مرتب اجرا بشه.
 - **هزینه اجرای الگوریتم چقدره؟**
 - الگوریتم تشخیص هزینه محاسباتی بالایی داره ($O(n^2)$ یا بیشتر).
 - هر چه تعداد پردازنده‌ها بیشتر باشه → اجرای مکرر گرون‌تر میشه.

۲. بازیابی از بن‌بست

- وقتی بن‌بست شناسایی شد، سیستم باید بعضی پردازنده‌ها رو **Rollback (برگردوندن/خاتمه دادن)** کنه.
- نکته:
 - در هر **چرخه مستقل (disjoint cycle)** حداقل باید یک پردازنده خاتمه داده بشه.
 - پس اگه گراف منابع چند چرخه داشته باشه → چندین پردازنده باید قربانی بشن.

۳. مشکل اجرای دلخواه (Arbitrary Invocation)

- اگه الگوریتم **به طور نامنظم یا دیر** فراخوانی بشه:
 - ممکنه گراف منابع شامل چندین چرخه بشه.
 - در این حالت معلوم نیست **کدوم پردازنده‌ها مقصر اصلی** هستن.
 - در نتیجه مجبور می‌شیم چندین پردازنده رو بگشیم (حتی شاید بیش از حد لازم).

جمع‌بندی

- اجرای خیلی کم → ممکنه بن‌بست دیر کشف بشه و چندین چرخه تشکیل بشه.
- اجرای خیلی زیاد → هزینه محاسباتی بالا و افت کارایی.
- بهترین راهکار: **یک توازن بین هزینه اجرا و ریسک بن‌بست.**

Recovery from Deadlock: Process Termination

وقتی سیستم تشخیص داد که بن‌بست رخ داده، یکی از روش‌های بازیابی اینه که پردازنده‌ها (یا تردها) رو متوقف کنیم.

◆ روش‌های اصلی

1. همه‌ی پردازنده‌های درگیر بن‌بست Abort

- سریع‌ترین روش
- ولی خیلی **هزینه‌بر** (ممکنه کلی محاسبات از بین بره)

2. یکی‌یکی پردازنده‌ها تا چرخه بن‌بست از بین بره Abort

- کنترل بیشتری می‌ده
- هزینه کمتره، ولی زمان بیشتری می‌بره

◆ معیارهای انتخاب پردازنده برای Terminate

وقتی قراره یکی‌یکی پردازنده‌ها رو حذف کنیم، باید تصمیم بگیریم **کدوم پردازنده قربانی بشه**. معیارهای متداول:

1. Priority (اولویت پردازنده)

- پردازنده‌های کم‌اهمیت یا با اولویت پایین زودتر حذف بشن.
- 2. **چقدر محاسبه انجام داده و چقدر تا تکمیل باقی مونده؟**
 - پردازنده‌ای که تقریباً تمام شده، بهتره حذف نشه (هدر رفت بالاست).
- 3. **چه منابعی رو مصرف کرده؟**
 - اگه پردازنده منابع زیادی اشغال کرده باشه → حذفش منابع بیشتری آزاد می‌کنه.
- 4. **چه منابعی برای تکمیل نیاز داره؟**
 - اگه پردازنده هنوز به منابع زیادی نیاز داره → احتمالاً بهتره حذف بشه.
- 5. **چند پردازنده باید متوقف بشن؟**
 - تلاش برای کم کردن تعداد قربانی‌ها.
- 6. **نوع پردازنده (Batch یا Interactive)**
 - پردازنده‌های **Batch** معمولاً قابل حذف هستن،
 - پردازنده‌های **Interactive** حذفشون به تجربه کاربر لطمه می‌زنه.

جمع‌بندی

- **همه** → سریع ولی پرهزینه **Abort**.
- **انتخابی** → کاراتر، ولی نیاز به تصمیم‌گیری هوشمندانه **Abort**.
- انتخاب قربانی باید براساس **ترکیبی از معیارهای بالا** انجام بشه.

Recovery from Deadlock: Resource Preemption

به جای اینکه کل پردازش رو terminate کنیم، می‌تونیم منابع رو از پردازش‌ها پس بگیریم (preempt) و به بقیه بدیم.

◆ مراحل اصلی

1. Selecting a Victim (انتخاب قربانی)

- باید تصمیم بگیریم کدام پردازش منابعش گرفته بشه.
- ملاک: کمترین هزینه (پردازش‌ای که گرفتن منبع ازش کمترین خسارت رو بزنه).

2. Rollback (بازگردانی پردازش)

- وقتی منابع از پردازش‌های گرفته بشه، ممکنه نتونه ادامه بده.
- در این حالت باید برگردونده بشه به یک وضعیت امن قبلی (Safe State).
- بعد دوباره از اون نقطه اجرا میشه.

3. Starvation (گرسنگی پردازش‌ها)

- خطر این وجود داره که همیشه یک پردازش به‌عنوان قربانی انتخاب بشه.
- برای جلوگیری: تعداد دفعات rollback رو در انتخاب قربانی لحاظ می‌کنیم → یعنی پردازش‌ای که چند بار قربانی شده، دوباره انتخاب نشه.

✓ جمع‌بندی

- کردن کل پردازش‌هاست terminate انعطاف‌پذیرتر از Resource Preemption →
- اما پیچیده‌تره چون:
 - باید safe state نگه داریم،
 - باید جلوی starvation گرفته بشه.
- ممکنه هزینه‌بر باشه rollback،

End of Chapter 8 :)